

浙江大学



计算机组成与设计实验报告

项目名称: Lab30 RISC-V 微处理器设计

姓 名: 李维扬

学 号: 3230101883

学 院: 信息与工程学院

专 业: 电子科学与技术

指导老师: 唐奕、屈民军

2026 年 1 月 6 日

目 录

一	实验目的	2
二	实验任务与要求	2
	2.1 基本要求	2
	2.2 扩展要求	2
三	实验设备	2
四	实验内容	3
五	实验过程	3
	5.1 指令译码模块(ID)	3
	5.2 执行模块(EX)	19
	5.3 数据储存器模块(DataRAM)	26
	5.4 取指令级模块(IF)	27
	5.5 流水线寄存器设计	30
	5.6 CPU 顶层模块	34
六	实验结果	39
	6.1 仿真结果	39
	6.2 硬件测试结果	41
七	思考题	42

一 实验目的

- (1) 熟悉 RISC-V 指令系统。
- (2) 了解提高 CPU 性能的方法。
- (3) 掌握流水线 RISC-V 微处理器的工作原理。
- (4) 理解数据冒险、控制冒险的概念以及流水线冲突的解决方法。
- (5) 掌握流水线 RISC-V 微处理器的测试方法。
- (6) 了解用软件实现数字系统的方法。

二 实验任务与要求

2.1 基本要求

设计一个流水线 RISC-V 微处理器，具体要求如下所述。

- (1) 至少运行下列 RV32I 核心指令。
 - 算术运算指令： add, sub, addi
 - 逻辑运算指令： and, or, xor, slt, sltu, andi, ori, xori, slti, sltiu
 - 移位指令： sll, srl, sra, slli, srli, srai
 - 条件分支指令： beq, bne, blt, bge, bltu, bgeu
 - 无条件跳转指令： jal, jalr
 - 数据传送指令： lw, sw, lui, auipc
 - 空指令： nop
- (2) 采用 5 级流水线技术，对数据冒险实现转发或阻塞功能。
- (3) 在 Nexys Video 开发系统中实现微处理器，要求 CPU 的运行速度大于 25MHz。

2.2 扩展要求

- 要求设计的微处理器还能运行 lb, lh, lhu, lbu, lhu, lwu, sb, sh 或 sd 等字节、半字和双字数据传送指令。
- 要求设计的 CPU 增加异常（exception）、自陷（trap）、中断（interrupt）等处理方案。

三 实验设备

- 装有 Vivado 和 ModelSim SE 软件的计算机。
- Nexys Video 开发板一套。
- 带有 HDMI 接口的显示器一台。

四 实验内容

- (1) 从网络下载相关文件。
- (2) 编写指令译码单元 Decode 模块的 Verilog HDL 代码, 并用 ModelSim 进行功能仿真。
- (3) 编写寄存器堆 Register 模块的 Verilog HDL 代码。
- (4) 编写 ID 模块 Verilog HDL 代码, ID.v 文件已给端口列表。
- (5) 编写 ALU 模块的 Verilog HDL 代码并用 ModelSim 进行功能仿真。
- (6) 编写执行单元 EX 模块的 Verilog HDL 代码, EX.v 文件已给端口列表。
- (7) 编写 IF 模块的 Verilog HDL 代码并用 ModelSim 进行功能仿真。
- (8) 打开 Vivado 文件夹下的 Risc5CPU.xpr 工程, 生成符合 CPU 要求的数据存储器 IP 内核。
- (9) 编写 CPU 顶层的 Verilog HDL 代码, 并用 ModelSim 进行功能仿真。注意: 由于存在 IP 内核, 仿真时, 需加仿真库, 方法参考实验 3. 根据表 30.11 验证仿真结果。表格中显示的数据均为十六进制, 表格中“-”表示此处值无意义。
- (10) 再次打开 Vivado 文件夹下的 Risc5CPU.xpr 工程, 添加流水线 CPU 设计的全部代码, 然后综合、实现和下载至 Nexys Video 开发板。
- (11) 连接带有 HDMI 接口的显示器, 进行测试。首先将 SW0 置于低电平, 使 RISC-V CPU 工作在“单步”运行模式。复位后, 每按一下上边按键, RISC-V CPU 运行一步, 记录下显示器上的结果, 对照表 30.11 验证设计是否正确。

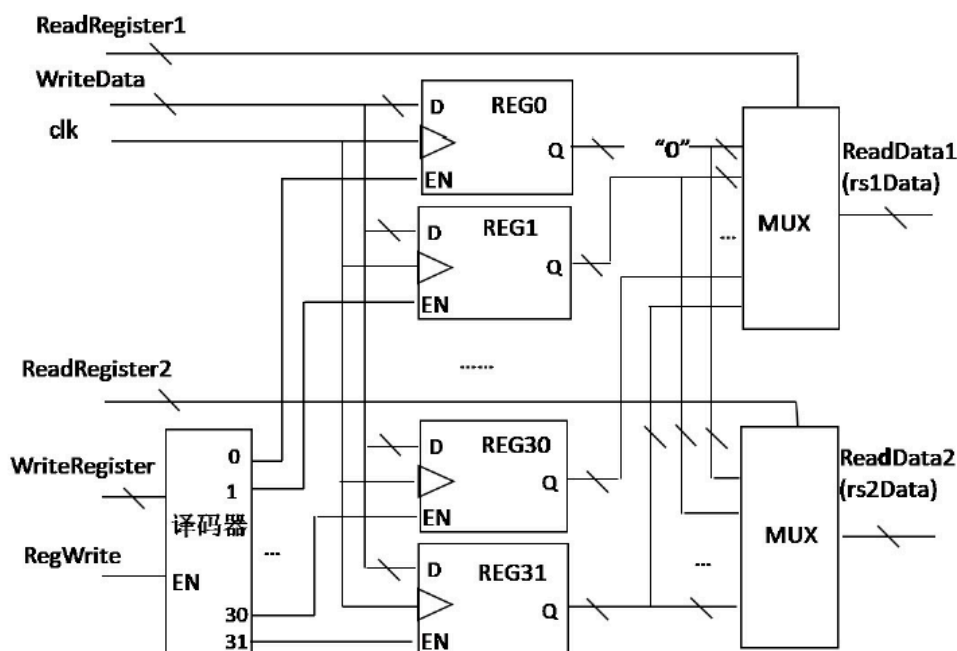
表 30.11 将在实验结果部分展示。

五 实验过程

5.1 指令译码模块(ID)

5.1.1 寄存器堆(Registers)子模块

寄存器堆由 32 个 32 位寄存器组成, 这些寄存器通过寄存器号进行读写存取。寄存器堆的原理框图如下图所示。因为读取寄存器不会更改其内容, 故只需提供寄存器号即可读出该寄存器内容。读取端口采用数据选择器即可实现读取功能。应注意的是, 0 号寄存器为常数 0。



对于往寄存器里写数据，需要目标寄存器号（WriteRegister）、待写入数据（WriteData）、写允许信号（RegWrite）三个变量。上图中 5 位二进制译码器完成地址译码，其输出控制目标寄存器的写使能信号 EN，决定将数据 WriteData 写入哪个寄存器。注意：用 Verilog HDL 设计描述寄存器堆时，用存储器变量定义 32 个 32 位寄存器更方便。下面为描述寄存器堆核心语句。

```

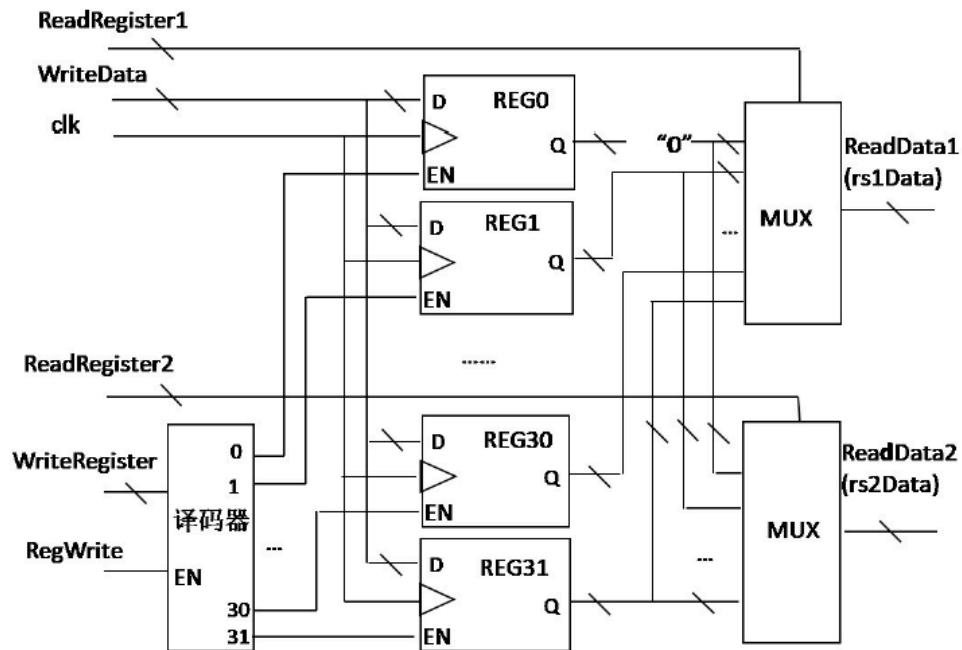
1 reg [31:0] regs [31:0]; // 定义 32*32 存储器变量
  assign ReadData1 = (ReadRegister1 == 5'b0) ? 32'b0 :
2   regs[ReadRegister1]; // 端口1读
  assign ReadData2 = (ReadRegister2 == 5'b0) ? 32'b0 :
3   regs[ReadRegister2]; // 端口2读
  always @ (posedge clk) if (RegWrite) regs[WriteRegister] <=
4   WriteData; // 数据写入

```

在流水线型 CPU 设计中，寄存器堆设计还应解决三阶数据相关的数据转发问题。当满足三阶数据相关条件时，寄存器具有 Read After Write 特性。设计时，只需要在上图设计寄存器堆的基础上添加少量电路就可实现 Read After Write 特性，如下图所示。图中的 RBW_Registers 模块就是实现的 Read Before Write 寄存器堆。图中转发检测电路的输出表达式为：

$$rs1Sel = RegWrite \&\&(WriteAddr! = 0)\&\&(WriteAddr == rs1Addr) \quad (30.1)$$

$$rs2Sel = RegWrite \&\&(WriteAddr! = 0)\&\&(WriteAddr == rs2Addr) \quad (30.2)$$



```

1  `timescale 1ns / 1ps
2  module register_32bit(
3      input wire clk,
4      input [4:0] ReadRegister1, ReadRegister2, //读取寄存器的地址
5      input [4:0] WriteRegister, //写入寄存器的地址
6      input RegWrite, //写使能信号
7      input [31:0] WriteData, //写入数据
8      output [31:0] ReadData1, ReadData2 //读取数据
9  );
10     reg[31:0] regs[31:0]; //定义32*32存储器变量
11
12
13     assign ReadData1 = (ReadRegister1==5'b0) ? 32'b0 :
regs[ReadRegister1]; //x0寄存器始终为0,端口1数据读出
14     assign ReadData2 = (ReadRegister2==5'b0) ? 32'b0 :
regs[ReadRegister2]; //x0寄存器始终为0,端口2数据读出
15     always @(posedge clk) begin //数据写入
16         if(RegWrite) begin
17             regs[WriteRegister] <= WriteData;
18         end
19     end
20 endmodule
21
22
23 module RAWregister(
24     input wire clk,
25     input [4:0] rs1Addr, rs2Addr, //读取寄存器的地址`
26     input [4:0] WriteAddr, //写入寄存器的地址
27     input RegWrite, //写使能信号
28     input [31:0] WriteData, //写入数据
29     output [31:0] rs1Data, rs2Data //读取数据
30 );
31     wire rs1Sel, rs2Sel;

```

```

32    assign rs1Sel = RegWrite && (WriteAddr != 5'b0) && (WriteAddr ==
rs1Addr); //30.1
33    assign rs2Sel = RegWrite && (WriteAddr != 5'b0) && (WriteAddr ==
rs2Addr); //30.2
34    wire [31:0] RBW_rdData1, RBW_rdData2;
35
36    register_32bit RBWregister(
37        .clk(clk),
38        .ReadRegister1(rs1Addr),
39        .ReadRegister2(rs2Addr),
40        .WriteRegister(WriteAddr),
41        .RegWrite(RegWrite),
42        .WriteData(WriteData),
43        .ReadData1(RBW_rdData1),
44        .ReadData2(RBW_rdData2)
45    );
46    assign rs1Data = rs1Sel ? WriteData : RBW_rdData1;
47    assign rs2Data = rs2Sel ? WriteData : RBW_rdData2;
48    endmodule

```

代码设计解析

寄存器堆部分首先用二维寄存器数组实现 32×32 通用寄存器文件，并对 x0 寄存器做特殊处理保证其始终为 0。

在此基础上 RAWregister 模块通过 rs1Sel、rs2Sel 控制信号在满足 RegWrite 且写地址与读地址相等时直接旁路写回数据，从而实现 Read-Before-Write 的数据前推，在不改变时钟结构的前提下降低了三阶数据相关引起的流水线停顿。

5.1.2 指令译码(Decode)及立即数产生电路子模块

Decode 模块包括译码部分，主要功能是对指令进行译码，将指令的操作码和立即数提取出来，还包括 ALUcode 的生成，用于 ALU 模块的控制和立即数产生电路的设计，其设计原理如下所示。RISC-V 将指令分为 R_type、I_type、S_type、U_type、SB_type、U_type 等六类。从电路设计角度看，根据操作数的来源和立即数构成方式不同，再次细分指令如下：

- R_type 类：操作码 (opcode, 简称 op) 为 7'h33，R 类的所有指令，两个操作数分别为 rs1 和 rs2；
- I_type 类：操作码 7'h13，I 类的算术逻辑运算指令和移位指令，两个操作数分别为 rs1 和立即数 imm；
- LW 指令：操作码 7'h03，I 类的数据传送指令 lw，两个操作数分别为 rs1 和立即数 imm；
- JALR 指令：操作码 7'h67，I 类的无条件分支指令 jalr，两个操作数分别为 PC 和常数 4；

- SW 指令：操作码 7'h23，S 类的数据传送指令 sw，两个操作数分别为 rs1 和立即数 imm；
- SB_type 类：操作码 7'h63，SB 类的所有指令，两个操作数分别为 PC 和立即数 imm；
- LUI 指令：操作码 7'h37，U 类的数据传送指令 lui，只有一个操作数（立即数 imm）；
- AUIPC 指令：操作码 7'h17，U 类的数据传送指令 auipc，两个操作数分别为 PC 和立即数 imm；
- JAL 指令：操作码 7'h6f，U 类的无条件分支指令 jal，两个操作数分别为 PC 和常数 4。

因此，设置 R_type、I_type、SB_type、LW、JALR、SW、LUI、AUIPC 和 JAL 等变量来表示指令类型，各变量的值由下式决定：

$$R_type = (op == R_type_op)$$

$$I_type = (op == I_type_op)$$

$$SB_type = (op == SB_type_op)$$

$$LW = (op == LW_op)$$

$$JALR = (op == JALR_op)$$

$$SW = (op == SW_op)$$

$$LUI = (op == LUI_op)$$

$$AUIPC = (op == AUIPC_op)$$

$$JAL = (op == JAL_op)$$

(1) 只有 LW 指令读取存储器且回写数据取自存储器，所以有：

$$MemtoRead = LW$$

$$MemRead = LW$$

(2) 只有 SW 指令会对存储器写数据，所以有

$$MemWrite = SW$$

(3) 需要进行回写的指令类型有 R_type、I_type、LW、JALR、LUI、AUIPC 和 JAL。所以有：

$$RegWrite = R_type \parallel I_type \parallel LW \parallel JALR \parallel LUI \parallel AUIPC \parallel JAL$$

(4) 只有 JALR 和 JAL 两条无条件分支指令，所以有：

$$Jump = JALR \parallel JAL$$

(5) 操作数 A 和 B 的选择信号的确定

分析各类指令，可得到表 30.4 操作数选择的功能表。

类型	ALUSrcA_id	ALUSrcB_id[1:0]	说明
R_type	0	2'b00	$rd=rs1 \text{ op } rs2$
I_type	0	2'b01	$rd=rs1 \text{ op } imm$
LW	0	2'b01	$rs1 + imm$
SW	0	2'b01	$rs1 + imm$
JALR	1	2'b10	$rd=pc + 4$
JAL	1	2'b10	$rd=pc + 4$
LUI	1'bx	2'b01	$rd=imm$
AUIPC	1	2'b01	$rd=pc + imm$

从上表可获得 ALUSrcA_id 和 ALUSrcB_id[1:0] 表达式：

$$ALUSrcA = JALR \parallel JAL \parallel AUIPC$$

$$ALUSrcB[1] = JAL \parallel JALR$$

$$ALUSrcB[0] = \sim (R_type \parallel JAL \parallel JALR)$$

(6) ALUCode 的确定

除了条件分支指令，其它指令都需要 ALU 执行运算，共有 11 种不同运算，ALUCode 信号需用 4 位二进制表示。最主要为加法运算，设为默认算法，ALUCode 的功能表如下所示。注意：表中 funct7[6] 与 funct6[5] 在指令中为同一位置，即 instruction[30]。

R_type	I_type	LUI	funct3	funct7[6] (funct6[5])	ALUCode	备注
1	0	0	3'o0	0	4'd0	加
1	0	0	3'o0	1	4'd1	减
1	0	0	3'o1	0	4'd6	左移 $A \ll B$
1	0	0	3'o2	0	4'd9	$A < B ? 1:0$
1	0	0	3'o3	0	4'd10	$A < B ? 1:0$ (无符号数)
1	0	0	3'o4	0	4'd4	异或
1	0	0	3'o5	0	4'd7	右移 $A \gg B$
1	0	0	3'o5	1	4'd8	算术右移 $A \ggg B$
1	0	0	3'o6	0	4'd5	或
1	0	0	3'o7	0	4'd3	与
0	1	0	3'o0	x	4'd0	加
0	1	0	3'o1	x	4'd6	左移
0	1	0	3'o2	x	4'd9	$A < B ? 1:0$
0	1	0	3'o3	x	4'd10	$A < B ? 1:0$ (无符号数)
0	1	0	3'o4	x	4'd4	异或
0	1	0	3'o5	0	4'd7	右移 $A \gg B$
0	1	0	3'o5	1	4'd8	算术右移 $A \ggg B$
0	1	0	3'o6	x	4'd5	或
0	1	0	3'o7	x	4'd3	与
0	0	1	x	x	4'd2	送数: $ALUResult=B$
其它					4'd0	加

(7) 立即数产生电路 (ImmGen) 设计

L_type、SB_type、LW、JALR、SW、LUI、AUIPC 和 JAL 这几类指令均用到立即数。由于 L_type 的算术逻辑运算与移位运算指令的立即数构成方法不同，这里再设定一个变量 Shift 来区分两者。Shift=1 表示移位运算，否则为算术逻辑运算。Shift 值由式 (30.11) 计算。

$$\text{Shift} = (\text{funct3} == 1) \parallel (\text{funct3} == 5) \quad (30.11)$$

立即数构成和扩展方法如下所示。

立即数 imm 产生方法

类别	Shift	Imm	说明
I_type	1	{27'd0,inst[24:20]}	无符号数 0 扩展
I_type	0	{20{inst[31]},inst[31:20]}	有符号数符号扩展
LW	x		
SW	x		
LUI	x	{inst[31:12], 12'd0}	
AUIPC	x		
other	x	-	

立即数 offset 产生方法

类别	offset	说明
JALR	{20{inst[31]},inst[31:20]}	有符号数 符号扩展
JAL	{11{inst[31]}, inst[31], inst[19:12], inst[20], inst[30:21], 1'b0}	
SB_type	{19{inst[31]}, inst[31], inst[7],inst[30:25], inst[11:8], 1'b0}	
other	-	

```

//
1 *****
2 // //
3 // Decode.v
4 //
5 *****
6 module Decode(
7     // Outputs
8     MemtoReg, RegWrite, MemWrite,
9     MemRead, ALUCode, ALUSrcA, ALUSrcB, Jump, JALR, Imm, offset,
10    // Inputs
11    Instruction);
12    input [31:0] Instruction; // current instruction
13    output      MemtoReg;    // use memory output as data to write into
14    register
15    output      RegWrite;    // enable writing back to the register
16    output      MemWrite;    // write to memory
17    output      MemRead;
18    output [3:0] ALUCode;     // ALU operation select
19    output      ALUSrcA;
20    output [1:0] ALUSrcB;
21    output      Jump;

```

```

20     output          JALR;
21     output[31:0]    Imm,offset;
22
23     //
24     *****
25     // instruction type decode
26     //
27     *****
28     parameter R_type_op= 7'b0110011;
29     parameter I_type_op= 7'b0010011;
30     parameter SB_type_op= 7'b1100011;
31     parameter LW_op= 7'b0000011;
32     parameter JALR_op= 7'b1100111;
33     parameter SW_op= 7'b0100011;
34     parameter LUI_op= 7'b0110111;
35     parameter AUIPC_op= 7'b0010111;
36     parameter JAL_op= 7'b1101111;
37     //
38     //
39     parameter ADD_func3 = 3'b000 ;
40     parameter SUB_func3 = 3'b000 ;
41     parameter SLL_func3 = 3'b001 ;
42     parameter SLT_func3 = 3'b010 ;
43     parameter SLTU_func3 = 3'b011 ;
44     parameter XOR_func3 = 3'b100 ;
45     parameter SRL_func3 = 3'b101 ;
46     parameter SRA_func3 = 3'b101 ;
47     parameter OR_func3 = 3'b110 ;
48     parameter AND_func3 = 3'b111;
49     //
50     parameter ADDI_func3 = 3'b000 ;
51     parameter SLLI_func3 = 3'b001 ;
52     parameter SLTI_func3 = 3'b010 ;
53     parameter SLTIU_func3 = 3'b011 ;
54     parameter XORI_func3 = 3'b100 ;
55     parameter SRLI_func3 = 3'b101 ;
56     parameter SRAI_func3 = 3'b101 ;
57     parameter ORI_func3 = 3'b101 ;
58     parameter ANDI_func3 = 3'b111;
59     //
60     parameter alu_add= 4'b0000;
61     parameter alu_sub= 4'b0001;
62     parameter alu_lui= 4'b0010;
63     parameter alu_and= 4'b0011;
64     parameter alu_xor= 4'b0100;
65     parameter alu_or = 4'b0101;
66     parameter alu_sll= 4'b0110;
67     parameter alu_srl= 4'b0111;
68     parameter alu_sra= 4'b1000;
69     parameter alu_slt= 4'b1001;
70     parameter alu_sltu= 4'b1010;
71     //
72     *****
73     // instruction field
74     //
75     *****
76     wire [6:0] op;

```

```

74  wire          funct76_65;
75  wire [2:0]    funct3;
76  assign op      = Instruction[6:0];
77  assign funct76_65 = Instruction[30];
78  assign funct3   = Instruction[14:12];
79
80  //细分指令类型
81  wire R_type, I_type, SB_type, LW, SW, LUI, AUIPC, JAL;
82  assign R_type = (op == R_type_op);
83  assign I_type = (op == I_type_op);
84  assign SB_type = (op == SB_type_op);
85  assign LW = (op == LW_op);
86  assign JALR = (op == JALR_op);
87  assign SW = (op == SW_op);
88  assign LUI = (op == LUI_op);
89  assign AUIPC = (op == AUIPC_op);
90  assign JAL = (op == JAL_op);
91  //只有LW读取内存且回写数据取自寄存器
92  assign MemtoReg = LW;
93  assign MemRead = LW;
94  //只有SW写内存
95  assign MemWrite = SW;
96  //需要写回的指令
97  assign RegWrite = R_type || I_type || LW || JALR || LUI || AUIPC ||
JAL;
98  //无条件跳转
99  assign Jump = JAL || JALR;
100 //操作数A和B的选择信号的确定, 来自表30.4
101 assign ALUSrcA = JALR || JAL || AUIPC;
102 assign ALUSrcB[1] = JAL || JALR;
103 assign ALUSrcB[0] = ~(R_type || JAL || JALR);
104
105 //ALUCode的确定, 来自表30.5
106 reg [3:0] ALUCode_temp;
107 always @(*) begin
108     casez ({R_type, I_type, LUI, funct3, funct76_65})
109         7'b100_000_0: ALUCode_temp = 4'd0; //加
110         7'b100_000_1: ALUCode_temp = 4'd1; //减
111         7'b100_001_0: ALUCode_temp = 4'd6; //左移
112         7'b100_010_0: ALUCode_temp = 4'd9; //A<B?1:0
113         7'b100_011_0: ALUCode_temp = 4'd10; //A<B?1:0 (无符号)
114         7'b100_100_0: ALUCode_temp = 4'd4; //异或
115         7'b100_101_0: ALUCode_temp = 4'd7; //右移
116         7'b100_101_1: ALUCode_temp = 4'd8; //算术右移
117         7'b100_110_0: ALUCode_temp = 4'd5; //或
118         7'b100_111_0: ALUCode_temp = 4'd3; //与
119     //Immediate
120         7'b010_000_?: ALUCode_temp = 4'd0; //加
121         7'b010_001_?: ALUCode_temp = 4'd6; //左移
122         7'b010_010_?: ALUCode_temp = 4'd9; //A<B?1:0
123         7'b010_011_?: ALUCode_temp = 4'd10; //A<B?1:0 (无符号)
124         7'b010_100_?: ALUCode_temp = 4'd4; //异或
125         7'b010_101_0: ALUCode_temp = 4'd7; //右移
126         7'b010_101_1: ALUCode_temp = 4'd8; //算术右移
127         7'b010_110_?: ALUCode_temp = 4'd5; //或
128

```

```

129         7'b010_111_?: ALUCode_temp = 4'd3; //与
130         7'b001_???_?: ALUCode_temp = 4'd2; //送数: ALUResult=B
131     //其它
132     default: ALUCode_temp = 4'd0; //加
133     endcase
134 end
135 assign ALUCode = ALUCode_temp;
136
137 //立即数
138 //设置Shift信号
139 wire Shift;
140 assign Shift=(funct3==3'b001)|| (funct3==3'b101);
141
142 reg [31:0] Imm_temp,offset_temp;
143 //Imm产生方法, 来自表30.6a
144 always @(*) begin
145     if (I_type==1 && Shift==1)begin
146         Imm_temp = { 27'd0, Instruction[24:20] }; //shift
147     end
148     else if (I_type==1 && Shift==0)begin
149         Imm_temp = { {20{Instruction[31]}}, Instruction[31:20] }; //
immediate
150     end
151     else if (LW==1) begin
152         Imm_temp = { {20{Instruction[31]}}, Instruction[31:20] }; //
load
153     end
154     else if (SW==1)begin
155         Imm_temp = { {20{Instruction[31]}}, Instruction[31:25],
Instruction[11:7] }; //store
156     end
157     else if (LUI==1)begin
158         Imm_temp = { Instruction[31:12], 12'd0 }; //LUI
159     end
160     else if (AUIPC==1)begin
161         Imm_temp = { Instruction[31:12], 12'd0 }; //AUIPC
162     end
163     else begin
164         Imm_temp = 32'b0;//default
165     end
166 end
167 //offset产生方法, 来自表30.6b
168 always @(*) begin
169     if (JALR==1) begin
170         offset_temp = { {20{Instruction[31]}},
Instruction[31:20] }; //JALR
171     end
172     else if (JAL==1) begin
173         offset_temp = { {11{Instruction[31]}}, Instruction[31],
Instruction[19:12], Instruction[20], Instruction[30:21], 1'b0 }; //JAL
174     end
175     else if (SB_type==1) begin
176         offset_temp = { {19{Instruction[31]}}, Instruction[31],
Instruction[7], Instruction[30:25], Instruction[11:8], 1'b0 }; //SB
177     end
178     else begin

```

```

179         offset_temp = 32'b0; //default
180     end
181 end
182
183
184     assign Imm=Imm_temp;
185     assign offset=offset_temp;
186
187 endmodule

```

代码设计解析

首先从指令的低 7 位提取 opcode、第 30 位提取 funct7[6]、第[14:12]位提取 funct3，这三个字段构成译码的主要依据。

随后与预定义的操作码常量比较生成 R_type、I_type、SB_type 等九个指令类型信号，每个信号仅在对应指令类型时为高，便于后续用组合逻辑直接推导控制信号。在控制信号生成部分，MemtoReg 和 MemRead 只在 LW 时置 1，MemWrite 只在 SW 时置 1，RegWrite 则涵盖所有需要写回寄存器的指令类型（R_type、I_type、LW、JALR、LUI、AUIPC、JAL），Jump 信号由 JAL 和 JALR 两类无条件跳转指令触发；操作数选择信号 ALUSrcA 在 PC 相关指令（JALR、JAL、AUIPC）时为 1 表示选 PC，ALUSrcB[1:0] 通过两级组合区分寄存器 rs2、立即数 Imm 和常数 4 三种来源，这套控制信号确保了各类指令在流水线后续级能以统一的数据通路执行。

ALUCode 的生成采用 casez 结构，将 R_type、I_type、LUI 三个类型标志与 funct3、funct7[6] 拼接成 7 位判断码，通过模式匹配为 11 种不同运算分配 4 位编码；特别地，LUI 指令编码为 alu_lui 使 ALU 直接输出 B 操作数，移位类 I 指令根据 funct7[6] 区分逻辑右移与算术右移，而默认情况统一为加法。

立即数生成分为 Imm 和 offset 两路：Imm 路径用 Shift 信号区分 I_type 中的移位指令（零扩展 5 位移位量）与算术逻辑指令（符号扩展 12 位立即数），同时处理 LW/SW 的 12 位偏移、LUI/AUIPC 的高 20 位拼接；offset 路径专门服务于跳转与分支，对 JALR 做符号扩展 12 位、对 JAL 按 UJ 格式重排 20 位并左移 1 位、对 SB_type 按 SB 格式重排 12 位并左移 1 位，末位补 0 保证跳转地址字对齐。

整个设计通过将指令格式差异集中在这两个 always 块中处理，使得后续 EX 级可以用统一的加法器完成地址计算，同时译码级的组合逻辑延迟被限制在 casez 查找表和立即数拼接路径上。

仿真验证

运行已经给出的 Decode 模块的仿真代码：



结果如图所示，该模块正常运行。

5.1.3 分支检测(Branch Test)电路

分支检测电路主要用于判断分支条件是否成立，在 Verilog HDL 可以用比较运算符“>”、“<”和“==”描述，但要注意符号数和无符号数的处理方法不同。在这里，我们用加法器来实现。

- (1) 用一个 32 位加法器完成 $rs1Data + (-rs2Data) + 1$ (即 $rs1Data - rs2Data$)，设结果为 $sum[31:0]$ 。
- (2) 确定比较运算的结果。对于比较运算来说，如果最高位不同，即 $rs1Data[31] \neq rs2Data[31]$ ，可根据 $rs1Data[31]$ 、 $rs2Data[31]$ 决定比较结果，但是应注意符号数、无符号数的最高位 $rs1Data[31]$ 、 $rs2Data[31]$ 代表意义不同。若两数最高位相同，则两数之差不会溢出，所以比较运算结果可由两个操作数之差的符号位 $sum[31]$ 决定。

在符号数比较运算中， $rs1Data < rs2Data$ 有以下两种情况：

- (1) $rs1Data$ 为负数， $rs2Data$ 为 0 或正数：

$$(rs1Data[31] \&\&(\sim rs2Data[31]))$$

- (2) $rs1Data$ 、 $rs2Data$ 符号相同， sum 为负：

$$(rs1Data[31] \sim rs2Data[31])\&\& sum[31]$$

因此，符号数 $rs1Data < rs2Data$ 比较运算结果为：

$$isLT = (rs1Data[31] \&\&(\sim rs2Data[31])) \parallel ((rs1Data[31] \sim rs2Data[31])\&\& sum[31])$$

同样地，无符号数比较运算中， $rs1Data < rs2Data$ 有以下两种情况：

- (1) $rs1Data$ 最高位为 0、 $rs2Data$ 最高位为 1：

$$(\sim rs1Data[31])\&\& rs2Data[31]$$

(2) rs1Data、rs2Data 最高位相同，sum 为负：

$$(rs1Data[31] \sim rs2Data[31]) \&\& sum[31]$$

因此，无符号数比较运算结果为：

$$isLTU = ((\sim rs1Data[31]) \&\& rs2Data[31]) \parallel ((rs1Data[31] \sim rs2Data[31]) \&\& sum[31])$$

最后用数据选择器，即可完成分支检测。

$$Branch = \begin{cases} \neg(sum[31 : 0]), & SB_type \wedge (funct3 == beq_funct3) \\ (sum[31 : 0]), & SB_type \wedge (funct3 == bne_funct3) \\ isLT, & SB_type \wedge (funct3 == blt_funct3) \\ \neg isLT, & SB_type \wedge (funct3 == bge_funct3) \\ isLTU, & SB_type \wedge (funct3 == bltu_funct3) \\ \neg isLTU, & SB_type \wedge (funct3 == bgeu_funct3) \\ 0, & others \end{cases}$$

```

1  module BranchTest (
2      input wire [31:0] rs1Data,
3      input wire [31:0] rs2Data,
4      input wire SB_type,
5      input wire [2:0] funct3,
6      output wire Branch
7  );
8      // 比较两个操作数的大小
9      wire isLT, isLTU;
10     wire [31:0] sum;
11     assign sum = rs1Data + ~rs2Data + 1;
12     assign isLT = rs1Data[31] && (~rs2Data[31]) || (rs1Data[31] ~^
rs2Data[31]) && sum[31];
13     assign isLTU = (~rs1Data[31]) && rs2Data[31] || (rs1Data[31] ~^
rs2Data[31]) && sum[31];
14     reg Branch_temp;
15
16     // 定义 funct3 的值
17     localparam beq_funct3 = 3'b000;
18     localparam bne_funct3 = 3'b001;
19     localparam blt_funct3 = 3'b100;
20     localparam bge_funct3 = 3'b101;
21     localparam bltu_funct3 = 3'b110;
22     localparam bgeu_funct3 = 3'b111;
23     // 分支检测 (30.14)
24     always @(*) begin
25         if (SB_type == 1 && funct3 == beq_funct3) begin
26             Branch_temp = ~(sum[31:0]);
27         end
28         else if (SB_type == 1 && funct3 == bne_funct3) begin
29             Branch_temp = sum[31:0];
30         end
31         else if (SB_type == 1 && funct3 == blt_funct3) begin

```



```

32         Branch_temp = isLT;
33     end
34     else if (SB_type == 1 && funct3 == bge_funct3) begin
35         Branch_temp = ~isLT;
36     end
37     else if (SB_type == 1 && funct3 == bltu_funct3) begin
38         Branch_temp = isLTU;
39     end
40     else if (SB_type == 1 && funct3 == bgeu_funct3) begin
41         Branch_temp = ~isLTU;
42     end
43     else begin
44         Branch_temp = 0;
45     end
46 end
47 // 赋值输出信号
48 assign Branch = Branch_temp;
49 endmodule

```

代码设计解析

利用加法器计算 $rs1Data - rs2Data$ 得到 sum ，并从符号位组合出有符号比较结果 $isLT$ 和无符号比较结果 $isLTU$ ，在此基础上根据 SB_type 和 $funct3$ 区分六类分支指令，对应生成是否跳转的 $Branch$ 信号。由于整个比较过程完全是组合逻辑， $Branch$ 信号可以在 ID/EX 之间尽早形成，与分支偏移量一起参与 PC 选择，从而兼顾了控制路径的时序和硬件资源的复用。

5.1.4 冒险检测(Hazard Detector)功能电路

由前面分析可知，冒险成立的条件为：

- (1) 上一条指令必须是 lw 指令($MemRead_ex=1$);
- (2) 两条指令读写同一个寄存器 ($rdAddr_ex = rs1Addr_id$ 或 $rdAddr_ex = rs2Addr_id$)。

当冒险成立应清空 ID/EX 寄存器并且阻塞流水线 ID 级、IF 级流水线，所以有：

$$Stall = ((rdAddr_ex == rs1Addr_id) \parallel (rdAddr_ex == rs2Addr_id)) \&\& MemRead_ex$$

$$IFWrite = \sim Stall$$

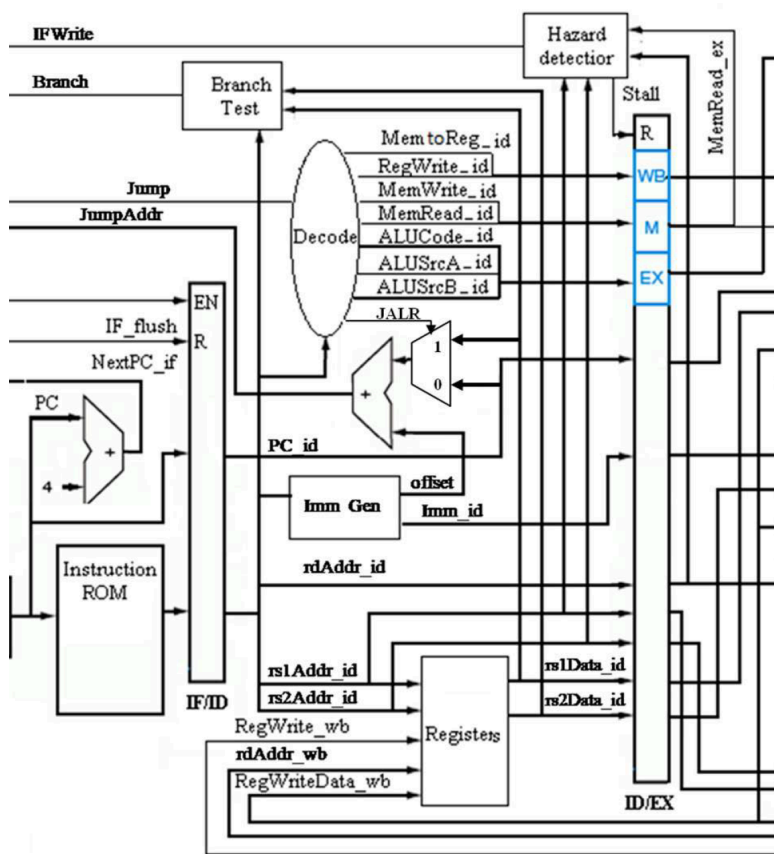
在用 VerilogHDL 描述 ID 模块时，冒险检测功能电路 (Hazard Detector) 等功能单元比较简单，直接在 ID 顶层描述。

5.1.5 ID 顶层

已经给出 ID 顶层接口如下：

引脚名称	方向	说明
clk	Input	系统时钟
Instruction_id[31:0]		指令机器码
PC_id[31:0]		指令指针
RegWrite_wb		寄存器写允许信号，高电平有效
rdAddr_wb[4:0]		寄存器的写地址。
RegWriteData_wb[31:0]		写入寄存器的数据
MemRead_ex		冒险检测的输入
rdAddr_ex[4:0]		
MemtoReg_id	Output	决定回写的数据来源（0：ALU；1：存储器）
RegWrite_id		寄存器写允许信号，高电平有效
MemWrite_id		存储器写允许信号，高电平有效
MemRead_id		存储器读允许信号，高电平有效
ALUCode_id[3:0]		决定 ALU 采用何种运算
ALUSrcA_id		决定 ALU 的 A 操作数的来源（0：rs1；1：pc）
ALUSrcB_id[1:0]		决定 ALU 的 B 操作数的来源(2'b00: rs2; 2'b01: imm; 2'b10: 常数 4)
Stall		ID/EX 寄存器清空信号，高电平表示插入一个流水线气泡
Branch		条件分支指令的判断结果，高电平有效
Jump		无条件分支指令的判断结果，高电平有效
IFWrite		阻塞流水线的信号，低电平有效
BranchAddr[31:0]		分支地址
Imm_id[31:0]		立即数
rdAddr_id[4:0]		回写寄存器地址
rs1Addr_id[4:0]		两个数据寄存器地址
rs2Addr_id[4:0]		
rs1Data_id[31:0]		寄存器两个端口输出数据
rs2Data_id[31:0]		

参考总电路图，ID 模块总体电路如下：



最终设计 ID 顶层如下：

```

1  module ID(clk, Instruction_id, PC_id, RegWrite_wb, rdAddr_wb,
2  RegWriteData_wb, MemRead_ex,
3      rdAddr_ex, MemtoReg_id, RegWrite_id, MemWrite_id, MemRead_id,
4  ALUCode_id,
5      ALUSrcA_id, ALUSrcB_id, Stall, Branch, Jump, IFWrite,
6  JumpAddr, Imm_id,
7      rs1Data_id, rs2Data_id, rs1Addr_id, rs2Addr_id, rdAddr_id);
8      input clk;
9      input [31:0] Instruction_id;
10     input [31:0] PC_id;
11     input RegWrite_wb;
12     input [4:0] rdAddr_wb;
13     input [31:0] RegWriteData_wb;
14     input MemRead_ex;
15     input [4:0] rdAddr_ex;
16     output MemtoReg_id;
17     output RegWrite_id;
18     output MemWrite_id;
19     output MemRead_id;
20     output [3:0] ALUCode_id;
21     output ALUSrcA_id;
22     output [1:0] ALUSrcB_id;
23     output Stall;
24     output Branch;
25     output Jump;
26     output IFWrite;
27     output [31:0] JumpAddr;
28     output [31:0] Imm_id;
29     output [31:0] rs1Data_id;
30     output [31:0] rs2Data_id;
31     output [4:0] rs1Addr_id, rs2Addr_id, rdAddr_id;
32
33     //内部信号定义
34     wire JALR;
35     wire [31:0] offset_id;
36     wire SB_type_id;
37     wire [2:0] funct3_id;
38     assign SB_type_id = (Instruction_id[6:0] == 7'b1100011);
39     assign funct3_id = Instruction_id[14:12];
40
41     assign rs1Addr_id = Instruction_id[19:15];
42     assign rs2Addr_id = Instruction_id[24:20];
43     assign rdAddr_id = Instruction_id[11:7];
44
45     //实例化RAW寄存器堆模块
46     RAWregister registers(
47         .clk(clk),
48         .rs1Addr(rs1Addr_id),
49         .rs2Addr(rs2Addr_id),
50         .WriteAddr(rdAddr_wb),
51         .RegWrite(RegWrite_wb),
52         .WriteData(RegWriteData_wb),
53         .rs1Data(rs1Data_id),
54         .rs2Data(rs2Data_id)
55     );
56
57     //实例化译码模块
58     Decode Decode_inst(

```

```

54         .Instruction(Instruction_id),
55         .JALR(JALR),
56         .MemtoReg(MemtoReg_id),
57         .RegWrite(RegWrite_id),
58         .MemWrite(MemWrite_id),
59         .MemRead(MemRead_id),
60         .ALUCode(ALUCode_id),
61         .ALUSrcA(ALUSrcA_id),
62         .ALUSrcB(ALUSrcB_id),
63         .Imm(Imm_id),
64         .Jump(Jump),
65         .offset(offset_id)
66     );
67     //实例化分支测试模块
68     BranchTest BranchTest_inst(
69         .rs1Data(rs1Data_id),
70         .rs2Data(rs2Data_id),
71         .SB_type(SB_type_id),
72         .funct3(funct3_id),
73         .Branch(Branch)
74     );
75     //计算跳转地址
76     assign JumpAddr = (JALR == 1) ? (rs1Data_id + offset_id) : (PC_id +
offset_id);
77     //冒险检测电路
78     assign Stall = ((rdAddr_ex == rs1Addr_id) || (rdAddr_ex ==
rs2Addr_id)) && MemRead_ex && (rdAddr_ex != 5'b0);
79     assign IFWrite = ~Stall;
80
81
82
83     endmodule

```

代码设计解析

通过 RAWregister 实现寄存器堆与写回级的紧耦合，先完成读寄存器和三阶数据前推，再由 Decode 模块产生控制信号和立即数、偏移量，同时将 rs1/rs2/rd 地址和 PC 等信息送入后级。

BranchTest 根据解码出的 SB_type 和 funct3 在 ID 级及早判断分支是否成立，结合 JALR/JAL 产生的 Jump 和 offset 计算 JumpAddr，从而尽量减少错误取指带来的流水线冲刷。Stall 与 IFWrite 逻辑在检测到 load-use 冒险且 $rd \neq x0$ 时阻塞前两级，实现了对数据冒险的硬件屏蔽。

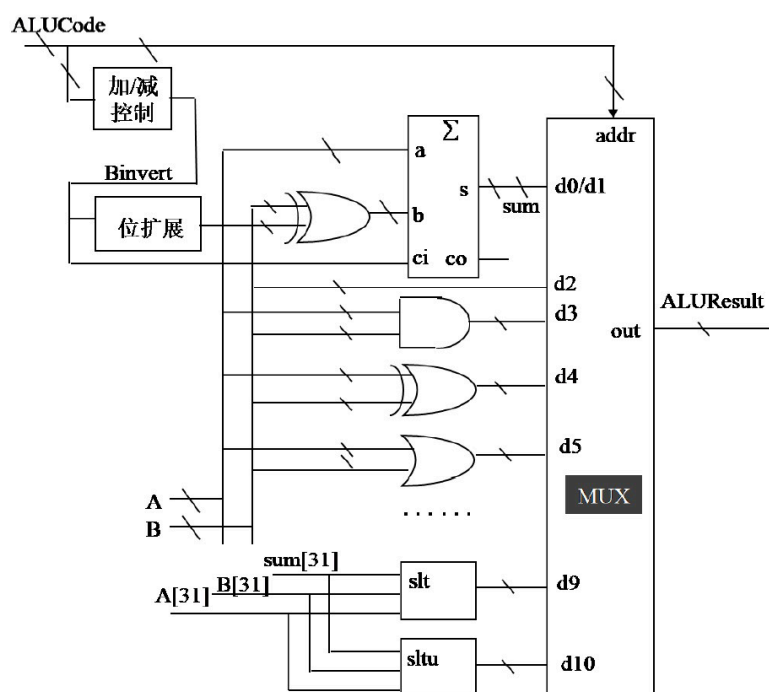
5.2 执行模块(EX)

5.2.1 ALU 子模块

算术逻辑运算单元 (ALU) 提供 CPU 的基本运算能力，如加、减、与、或、比较、移位等。具体而言，ALU 输入为两个操作数 A、B 和控制信号 ALUCode，由控制信号 ALUCode 决定采用何种运算，运算结果为 ALUResult。ALU 的功能表如下图所示。

ALUCode	ALUResult
4'b0000	$A + B$
4'b0001	$A - B$
4'b0010	B
4'b0011	$A \& B$
4'b0100	$A \wedge B$
4'b0101	$A \vee B$
4'b0110	$A \ll B$
4'b0111	$A \gg B$
4'b1000	$A \ggg B$
4'b1001	$A < B ? 1 : 0$, 其中 A、B 为有符号数
4'b1010	$A < B ? 1 : 0$, 其中 A、B 为无符号数

如图所示，ALU 需执行多种运算，为了提高运算速度，本设计可同时进行各种运算，再根据 ALUCode 信号选出所需结果。ALU 的基本结构如下图所示。



加、减电路的设计考虑：

减法、比较 (slt、sltu) 均可用加法器和必要辅助电路来实现。图 30.10 中的 Binvert 信号控制加减运算：若 Binvert 信号为低电平，则实现加法运算： $\text{sum} = A + B$ ；若 Binvert 信号为高电平，则电路为减法运算 $\text{sum} = A - B$ 。除加法外，减法、比较和分支指令都应使电路工作在减法状态，所以：

$$\text{Binvert} = \sim (\text{ALUCode} == 0)$$

最后要强调的是, 32 位加法器的运算速度决定了 RISC-V 微处理器的时钟信号频率的高低, 因此设计一个高速的 32 位加法器尤为重要。32 位加法器可采用实验 8 介绍的进位选择加法器。

比较电路的设计考虑:

比较电路的设计方法已经在分支检测电路介绍, 参考式 (30.12) 和式 (30.13) 可确定 slt 和 sltu 两条件的比较结果。

算术右移运算电路的设计考虑:

算术右移对有符号数而言, 移出的高位补符号位而不是 0。每右移一位相当于除以 2。例如, 有符号数负数 101001100(-76) 算术右移两位结果为 111010001(-19), 正数 01100111(103) 算术右移一位结果为 000110011(51)。Verilog HDL 的算术右移的运算符是“»”。

要实现算术右移应注意, 被移位对象必须定义是 reg 类型, 但是在 sra 指令, 被移位的对象操作数 A 为输入信号, 不能定义为 reg 类型。因此, 必须引入 reg 类型中间变量 A_reg, 相应的 Verilog HDL 语句为

```
1 reg signed[31:0] A_reg;
2 always @(*) begin A_reg = A; end
```

引入 reg 类型的中间变量 A_reg 后, 就可对 A_reg 进行算术右移操作。

最终设计 ALU 模块如下:

```
1 module ALU (
2     // Outputs
3     ALUResult,
4     // Inputs
5     ALUCode, A, B);
6 input [3:0] ALUCode;           // Operation select
7 input [31:0] A, B;
8 output [31:0] ALUResult;
9
10 // Decoded ALU operation select (ALUOpSel) signals
11 parameter alu_add= 4'b0000;
12 parameter alu_sub= 4'b0001;
13 parameter alu_lui= 4'b0010;
14 parameter alu_and= 4'b0011;
15 parameter alu_xor= 4'b0100;
16 parameter alu_or = 4'b0101;
17 parameter alu_sll= 4'b0110;
18 parameter alu_srl= 4'b0111;
19 parameter alu_sra= 4'b1000;
20 parameter alu_slt= 4'b1001;
21 parameter alu_sltu= 4'b1010;
22
23 //定义内部信号
```

```

24    wire Binvert;
25    wire [31:0] b, sum, d3, d4, d5, d6, d7, d8, d9, d10;
26    reg signed [31:0] A_reg;
27    reg [31:0] out;
28    //加减控制
29    assign Binvert = ~(ALUCode==0);
30    //位拓展
31    assign b = {32{Binvert}} ^ B;
32    //加法器
33    adder_32bits adder_32bits_ALUinst (
34        .a(A),
35        .b(b),
36        .ci(Binvert),
37        .s(sum),
38        .co()
39    );
40
41    //    //d3 and
42    //    assign d3 = A & B;
43    //    //d4 xor
44    //    assign d4 = A ^ B;
45    //    //d5 or
46    //    assign d5 = A | B;
47    //    //d6 sll
48    //    assign d6 = A << B[4:0];
49    //    //d7 srl
50    //    assign d7 = A >> B[4:0];
51
52    //d8 sra
53    always @(*) begin
54        A_reg = A;
55    end
56    assign d8 = A_reg >>> B[4:0];
57    //d9 slt
58    assign d9 = A[31] && (~B[31]) || (A[31]^~B[31]) && sum[31];
59    //d10 sltu
60    assign d10 = (~A[31]) && B[31] || (A[31]^~B[31]) && sum[31];
61
62    //选择输出
63    always @(*) begin
64        case (ALUCode)
65            alu_add, alu_sub: out = sum;
66            alu_lui:         out = B;
67            alu_and:         out = A & B;
68            alu_xor:         out = A ^ B;
69            alu_or:          out = A | B;
70            alu_sll:         out = (A << B[4:0]);
71            alu_srl:         out = (A >> B[4:0]);
72            alu_sra:         out = d8;
73            alu_slt:         out = d9;
74            alu_sltu:        out = d10;
75            default:         out = sum;
76        endcase
77    end
78    //赋值输出
79    assign ALUResult = out;
80 endmodule

```

代码设计解析

ALU 内部以加法器为核心，通过 Binvert 控制信号和按位取反加一，实现加法、减法和比较等运算的统一实现，并复用 sum 的符号位构造有符号、无符号比较结果。

算术右移部分先将输入 A 复制到有符号寄存器 A_reg，再使用 >>> 运算保证符号位扩展正确，而其它与、或、异或和移位等操作则由组合逻辑直接完成，最后为了简化代码结构，用 case 函数根据 ALUCode 在各运算结果中选择输出。在此过程中部分运算定义了中间变量以提高可读性。

仿真结果

运行已经给出的 ALU 模块的仿真代码：



运行结果如图所示，说明 ALU 模块正常工作。

5.2.2 数据前推电路

操作数 A 和 B 分别由数据选择器决定，数据选择器地址信号 ForwardA、ForwardB 的含义如图所示。

地址	操作数来源	说明
ForwardA=2'b00	rs1Data_ex	操作数 A 来自寄存器堆
ForwardA=2'b01	RegWriteData_wb	操作数 A 来自二阶数据相关的转发数据
ForwardA=2'b10	ALUResult_mem	操作数 A 来自一阶数据相关的转发数据
ForwardB=2'b00	rs2Data_ex	操作数 B 来自寄存器堆
ForwardB=2'b01	RegWriteData_wb	操作数 B 来自二阶数据相关的转发数据
ForwardB=2'b10	ALUResult_mem	操作数 B 来自一阶数据相关的转发数据

由前面介绍的一、二阶数据相关判断条件，不难得到：

$$\text{ForwardA}[0] = \text{RegWrite_wb} \ \&(\text{rdAddr_wb} \neq 0) \ \&(\text{rdAddr_mem} \neq \text{rs1Addr_ex}) \\ \&(\text{rdAddr_wb} == \text{rs1Addr_ex})$$

$$\text{ForwardA}[1] = \text{RegWrite_mem} \ \&(\text{rdAddr_mem} \neq 0) \ \&(\text{rdAddr_mem} == \text{rs1Addr_ex})$$

$$\text{ForwardB}[0] = \text{RegWrite_wb} \ \&(\text{rdAddr_wb} \neq 0) \ \&(\text{rdAddr_mem} \neq \text{rs2Addr_ex}) \\ \&(\text{rdAddr_wb} == \text{rs2Addr_ex})$$

$$\text{ForwardB}[1] = \text{RegWrite_mem} \ \&(\text{rdAddr_mem} \neq 0) \ \&(\text{rdAddr_mem} == \text{rs2Addr_ex})$$

这一部分直接在顶层中集成即可。

5.2.3 EX 顶层

EX 模块顶层接口定义如下：

引脚名称	方向	说明
ALUCode_ex[3:0]	Input	决定 ALU 采用何种运算
ALUSrcA_ex		决定 ALU 的 A 操作数的来源 (rs1、PC)
ALUSrcB_ex[1:0]		决定 ALU 的 B 操作数的来源(rs2、imm 和常数 4)
Imm_ex[31:0]		立即数
rs1Addr_ex[4:0]		rs1 寄存器地址
rs2Addr_ex[4:0]		rs2 寄存器地址
rs1Data_ex[31:0]		rs1 寄存器数据
rs2Data_ex[31:0]		rs2 寄存器数据
PC_ex[31:0]		指令指针
RegWriteData_wb[31:0]		写入寄存器的数据
ALUResult_mem[31:0]		ALU 输出数据
rdAddr_mem[4:0]		寄存器的写地址
rdAddr_wb[4:0]		
RegWrite_mem		寄存器写允许信号
RegWrite_wb		
ALUResult_ex[31:0]	Output	ALU 运算结果
MemWriteData_ex[31:0]		存储器的回写数据
ALU_A [31:0]		ALU 操作数，测试时使用
ALU_B [31:0]		

根据接口定义、电路图，最终设计如下代码：

```

1  `timescale 1ns / 1ps
2  module EX(ALUCode_ex, ALUSrcA_ex, ALUSrcB_ex, Imm_ex, rs1Addr_ex,
3  rs2Addr_ex, rs1Data_ex,
4  rs2Data_ex, PC_ex, RegWriteData_wb, ALUResult_mem, rdAddr_mem,
5  rdAddr_wb,
6  RegWrite_mem, RegWrite_wb, ALUResult_ex, MemWriteData_ex, ALU_A,
7  ALU_B);
8      input [3:0] ALUCode_ex;
9      input ALUSrcA_ex;
10     input [1:0] ALUSrcB_ex;
11     input [31:0] Imm_ex;
12     input [4:0] rs1Addr_ex;
13     input [4:0] rs2Addr_ex;
14     input [31:0] rs1Data_ex;
15     input [31:0] rs2Data_ex;
16     input [31:0] PC_ex;
17     input [31:0] RegWriteData_wb;
18     input [31:0] ALUResult_mem;
19     input [4:0] rdAddr_mem;
20     input [4:0] rdAddr_wb;
21     input RegWrite_mem;

```

```

19     input RegWrite_wb;
20     output [31:0] ALUResult_ex;
21     output [31:0] MemWriteData_ex;
22     output [31:0] ALU_A;
23     output [31:0] ALU_B;
24
25     //数据前推电路
26     wire [1:0] ForwardA, ForwardB;
27     assign ForwardA[0] = RegWrite_wb && (rdAddr_wb != 0) &&
28     (rdAddr_mem != rs1Addr_ex) && (rdAddr_wb == rs1Addr_ex);
29     assign ForwardA[1] = RegWrite_mem && (rdAddr_mem != 0) &&
30     (rdAddr_mem == rs1Addr_ex);
31     assign ForwardB[0] = RegWrite_wb && (rdAddr_wb != 0) &&
32     (rdAddr_mem != rs2Addr_ex) && (rdAddr_wb == rs2Addr_ex);
33     assign ForwardB[1] = RegWrite_mem && (rdAddr_mem != 0) &&
34     (rdAddr_mem == rs2Addr_ex);
35
36     //数据转发MUX结果
37     reg [31:0] Amux_out, Bmux_out;
38     always @(*) begin
39         case(ForwardA)
40             2'b00: Amux_out = rs1Data_ex;
41             2'b01: Amux_out = RegWriteData_wb;
42             2'b10: Amux_out = ALUResult_mem;
43             2'b11: Amux_out = ALUResult_mem;
44             default: Amux_out = rs1Data_ex;
45         endcase
46     end
47
48     always @(*) begin
49         case(ForwardB)
50             2'b00: Bmux_out = rs2Data_ex;
51             2'b01: Bmux_out = RegWriteData_wb;
52             2'b10: Bmux_out = ALUResult_mem;
53             2'b11: Bmux_out = ALUResult_mem;
54             default: Bmux_out = rs2Data_ex;
55         endcase
56     end
57
58     // ALU操作数MUX
59     assign ALU_A = ALUSrcA_ex ? PC_ex : Amux_out;
60     reg [31:0] ALU_B_reg;
61
62     always @(*) begin
63         case(ALUSrcB_ex)
64             2'b00: ALU_B_reg = Bmux_out;
65             2'b01: ALU_B_reg = Imm_ex;
66             2'b10: ALU_B_reg = 32'd4;
67             default: ALU_B_reg = Bmux_out;
68         endcase
69     end
70
71     assign ALU_B = ALU_B_reg;
72
73     assign MemWriteData_ex = Bmux_out; //MemWriteData_ex输出
74
75     //实例化ALU
76     ALU ALU_inst(
77         .ALUCode(ALUCode_ex),

```

```

72     .A(ALU_A),
73     .B(ALU_B),
74     .ALUResult(ALUResult_ex)
75 );
76 endmodule

```

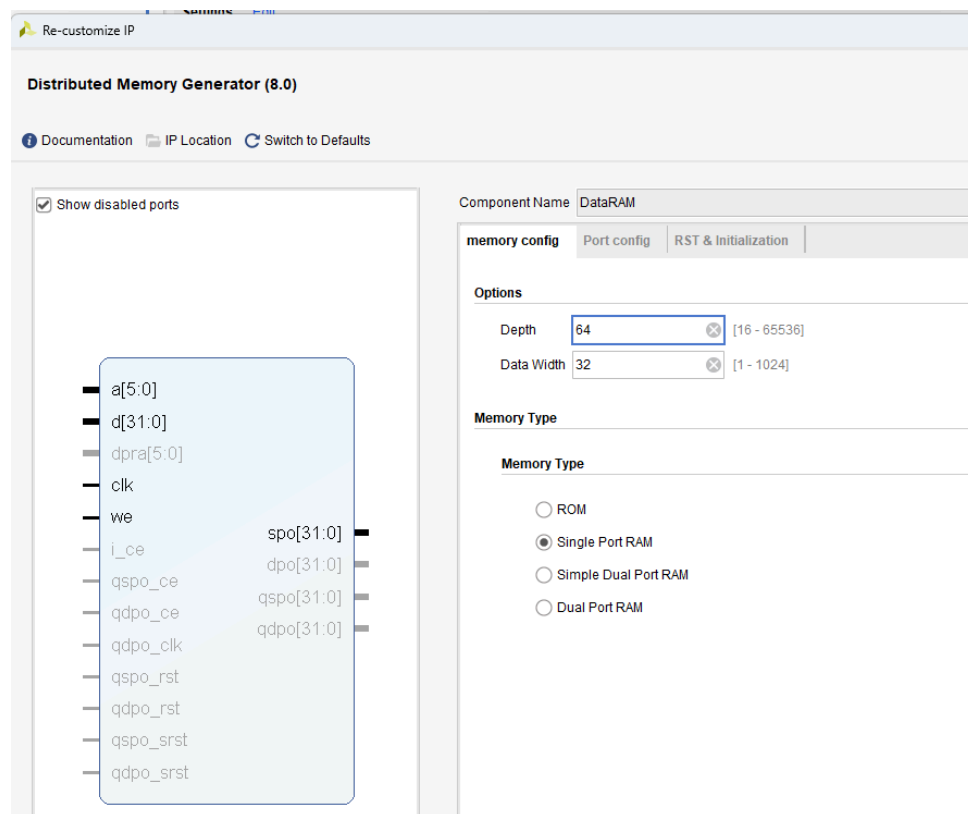
代码设计解析

首先依据 ForwardA 和 ForwardB 对 EX、MEM、WB 三级的写回结果进行数据前推，通过 Amux_out 和 Bmux_out 选择最新的操作数值，从而消除一、二阶数据相关。

再利用 ALUSrcA_ex 和 ALUSrcB_ex 在寄存器数据、Imm、PC 和常数 4 之间选择 ALU_A、ALU_B，实现普通算术逻辑运算与 PC 相对地址计算的统一；同时将 Bmux_out 直接作为 MemWriteData_ex 送往数据存储器，保证写内存总是使用经过前推后的最新 B 操作数。配合 ID_EX、EX_MEM 等流水线寄存器，这种前推结构保证大多数算术和逻辑相关都能在不插入气泡的情况下消除，而对于无法前推的 load-use 场景则依赖 ID 级 Stall 逻辑阻塞。

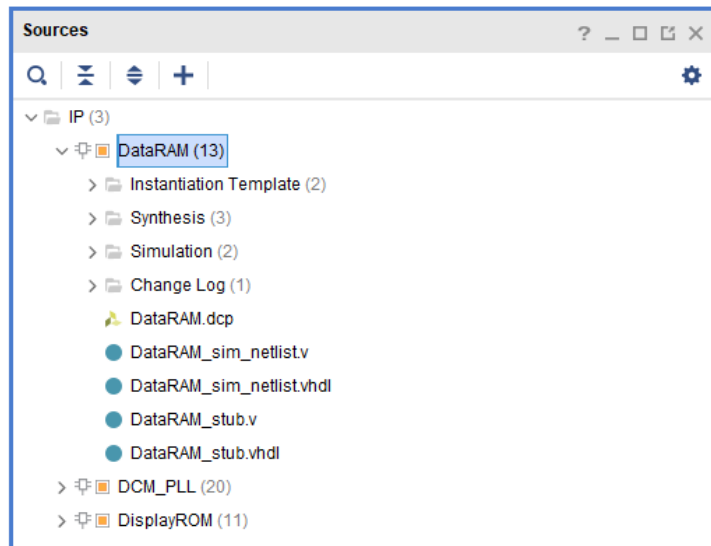
5.3 数据存储器模块(DataRAM)

数据存储器可用 Xilinx 的 IP 内核实现。考虑到 FPGA 的资源，数据存储器可设计为容量为 64×32 bit 的单端口 RAM，输出采用组合输出（Non Registered）。



直接在 vivado 工程中，通过 IP Catalog，选择 RAM，并设置参数，生成 IP 核。参数选择 32 位宽，64 个存储单元，single port RAM, non registered 输出。之后，直接在 CPU 顶层模块中实例化数据存储器模块即可。

定义成功后 source 界面如下：



5.4 取指令级模块(IF)

IF 模块由指令指针寄存器（PC）、指令存储器子模块（Instruction ROM）、指令指针选择器（MUX）和一个 32 位加法器组成，IF 模块接口信息如图所示。

引脚名称	方向	说明
clk	Input	系统时钟
reset		系统复位信号，高电平有效
Branch		条件分支指令的条件判断结果
Jump		无条件分支指令的条件判断结果
IFWrite		流水线阻塞信号
JumpAddr[31:0]		分支地址
Instruction [31:0]	Output	指令机器码
IF_flush		流水线清空信号
PC [31:0]		PC 值

指令存储器为组合存储器，可用 Verilog HDL 设计一个查找表阵列 ROM。考虑到 FPGA 的资源，该 ROM 容量可设计为 64×32bit 指令存储器模块 InstructionROM.v 已经提供，只需在顶层模块中实例化即可。内存放一段简单测试程序的机器码，对应的测试程序为：

```

1  /*-----
2      lui X30,0x3000
3      jalr X31 later(X0)
4  earlier:sw  X28, 0C(X0)

```

```

5      lw  X29, 04(X6)
6      slli X5, X29, 2          //数据冒险
7      lw  X28, 04(X6)
8      sltu X28,X6,X7
9  done:  jal X31,done
10 later: bne X0, X0, end        // 分支条件不成立
11        addi X5, X30, 42
12        add  X6, X0, X31
13        sub  X7, X5, X6        //操作A一阶数据相关, 操作B二阶数据相关
14        or   X28, X7, X5       //操作A一阶数据相关, 操作B三阶数据相关
15        beq  X0, X0, earlier   // 分支条件成立
16 end:    nop
17
18 -----*/
19 module InstructionROM(addr,dout);
20     input [5 : 0] addr;
21     output [31 : 0] dout;
22     reg [31 : 0] dout;
23     always @(*)
24         case (addr)
25             6'd0:  dout=32'h0000_3f37 ;//          lui X30,0x3000
26             6'd1:  dout=32'h0200_0fe7 ;//          jalr X31 later(X0)
27             6'd2:  dout=32'h01c0_2623 ;// earlier: sw  X28, 0C(X0)
28             6'd3:  dout=32'h0043_2e83 ;//          lw  X29, 4(X6)
29             6'd4:  dout=32'h002e_9293 ;//          sll  X5, X29, 2
30             6'd5:  dout=32'h0043_2e03 ;//          lw  X28, 4(X6)
31             6'd6:  dout=32'h0073_3e33 ;//          sltu X28,X6,X7
32             6'd7:  dout=32'h0000_0fef ;// done:   jal X31,done
33             6'd8:  dout=32'h0000_1c63 ;// later:  bne X0, X0, end        // 分
支条件不成立
34             6'd9:  dout=32'h042f_0293 ;//          addi X5, X30, 42
35             6'd10: dout=32'h01f0_0333 ;//          add  X6, X0, X31
36             6'd11: dout=32'h4062_83b3 ;//          sub  X7, X5, X6
37             6'd12: dout=32'h0053_ee33 ;//          or   X28, X7, X5
38             6'd13: dout=32'hfc00_0ae3 ;//          beq  X0, X0, earlier
39             6'd14: dout=32'h00000000 ;// end:    nop
40             default:dout=32'h00000000 ;//nop
41         endcase
42     endmodule

```

这些程序与表 30.11 的验证表格每一行对应。

5.4.1 IF 顶层

已知 IF 顶层接口如下：

引脚名称	方向	说明
clk	Input	系统时钟
reset		系统复位信号，高电平有效
Branch		条件分支指令的条件判断结果
Jump		无条件分支指令的条件判断结果
IFWrite		流水线阻塞信号
JumpAddr[31:0]		分支地址
Instruction [31:0]	Output	指令机器码
IF_flush		流水线清空信号
PC [31:0]		PC 值

设计 IF 模块如下所示：

```

1  `timescale 1ns / 1ps
2  module IF(clk, reset, Branch, Jump, IFWrite, JumpAddr, Instruction_if,
3  PC, IF_flush);
4      input clk;
5      input reset;
6      input Branch;
7      input Jump;
8      input IFWrite;
9      input [31:0] JumpAddr;
10     output [31:0] Instruction_if;
11     output [31:0] PC;
12     output IF_flush;
13
14     // PC寄存器
15     reg [31:0] PC_in, PC_out;
16     wire PCSource;
17     assign PCSource = Jump || Branch;
18     assign IF_flush = PCSource;
19     // 计算PC_in的值
20     always @(*) begin
21         if (PCSource) begin
22             PC_in = JumpAddr;
23         end else begin
24             PC_in = PC + 32'h00000004;
25         end
26     end
27     // 在时钟上升沿时触发
28     always @(posedge clk) begin
29         if (reset) begin
30             PC_out <= 32'b0; // 复位时将PC值重置为0
31         end else if (IFWrite) begin
32             PC_out <= PC_in; // 使能信号有效时更新PC值
33         end else begin
34             PC_out <= PC_out; // 使能信号无效时保持PC值不变
35         end
36     end
37     assign PC = PC_out;
38     //InstructionROM
39     InstructionROM InstructionROM_ints(
40         .addr(PC[7:2]),
41         .dout(Instruction_if)
42     );
43
44 endmodule

```

代码设计解析

IF 模块内部用 $PCSource = Jump || Branch$ 统一处理无条件跳转和条件分支，一旦分支或跳转成立即选择 $JumpAddr$ 作为下一条指令地址，并通过 IF_flush 清空后续级以减少控制冒险。

PC 寄存器在 reset 时清零，在 IFWrite 为高时才更新 PC_in，从而与 ID 级的 Stall 逻辑配合实现对取指阶段的阻塞。

最后通过 InstructionROM 以 PC[7:2]作为地址按字对齐读出指令，构成了简单清晰的取指流水级。IF_flush 与 IF_ID 寄存器的 reset 结合使用，可以在分支或跳转成立的那个时钟边沿将已经取到但错误的指令整体清空，只保留新路径上的指令流。

仿真结果

运行已经给出的 IF 模块的仿真代码：



可以看到所有指令被正常取出，说明该模块设计无误。

5.5 流水线寄存器设计

流水线寄存器负责将流水线的各部分分开，共有 IF/ID、ID/EX、EX/MEM、MEM/WB 四组，对四组流水线寄存器要求不完全相同，因此设计也有不同考虑。EX/MEM、MEM/WB 两组流水线寄存器只是普通的 D 型寄存器。当流水线发生数据冒险时，需要清空 ID/EX 流水线寄存器而插入一个气泡，因此 ID/EX 流水线寄存器是一个带同步清零功能的 D 型寄存器。当流水线发生数据冒险时，需要阻塞 IF/ID 流水线寄存器；若跳转指令或分支成立，则还需要清空 ID/EX 流水线寄存器。因此，IF/ID 流水线寄存器除同步清零功能外，还需要具有保持功能（即具有使能 EN 信号输入）。

设计流水线寄存器模块代码如下：

```
1 // IFID寄存器模块
2 module IF_ID (
3     input clk,                // 时钟信号
4     input reset,              // 复位信号，高电平有效
5     input EN,                 // 使能信号
6     input [31:0] Instruction_if, // 输入指令
7     input [31:0] PC_if,       // 输入PC值
8     output reg [31:0] Instruction_id, // 输出指令
9     output reg [31:0] PC_id    // 输出PC值
```

```

10 );
11
12 // 在时钟上升沿时触发
13 always @(posedge clk) begin
14     if (reset) begin
15         Instruction_id <= 32'b0; // 复位
16         PC_id <= 32'b0;
17     end else if (EN) begin
18         Instruction_id <= Instruction_if; // 使能信号有效时更新
19         PC_id <= PC_if;
20     end else begin
21         Instruction_id <= Instruction_id; // 使能信号无效时保持
22         PC_id <= PC_id;
23     end
24 end
25
26 endmodule
27
28 // IDEX寄存器模块
29 module ID_EX (
30     input clk, // 时钟信号
31     input reset, // 复位信号, 高电平有效
32     input [31:0] PC_id, // 输入PC值
33     input [31:0] rs1Data_id, // 输入源寄存器1数据
34     input [31:0] rs2Data_id, // 输入源寄存器2数据
35     input [31:0] Imm_id, // 输入立即数
36     input [4:0] rdAddr_id, // 输入目的寄存器地址
37     input [4:0] rs1Addr_id, // 输入源寄存器1地址
38     input [4:0] rs2Addr_id, // 输入源寄存器2地址
39     input [3:0] ALUCode_id, // 输入ALU控制信号
40     input ALUSrcA_id, // 输入ALU源操作数A选择信号
41     input [1:0] ALUSrcB_id, // 输入ALU源操作数B选择信号
42     input MemRead_id, // 输入内存读信号
43     input MemWrite_id, // 输入内存写信号
44     input RegWrite_id, // 输入寄存器写信号
45     input MemtoReg_id, // 输入内存到寄存器信号
46
47     output reg [31:0] PC_ex, // 输出PC值
48     output reg [31:0] rs1Data_ex, // 输出源寄存器1数据
49     output reg [31:0] rs2Data_ex, // 输出源寄存器2数据
50     output reg [31:0] Imm_ex, // 输出立即数
51     output reg [4:0] rdAddr_ex, // 输出目的寄存器地址
52     output reg [4:0] rs1Addr_ex, // 输出源寄存器1地址
53     output reg [4:0] rs2Addr_ex, // 输出源寄存器2地址
54     output reg [3:0] ALUCode_ex, // 输出ALU控制信号
55     output reg ALUSrcA_ex, // 输出ALU源操作数A选择信号
56     output reg [1:0] ALUSrcB_ex, // 输出ALU源操作数B选择信号
57     output reg MemRead_ex, // 输出内存读信号
58     output reg MemWrite_ex, // 输出内存写信号
59     output reg RegWrite_ex, // 输出寄存器写信号
60     output reg MemtoReg_ex, // 输出内存到寄存器信号
61
62 );
63
64
65

```



```

66 // 在时钟上升沿时触发
67 always @(posedge clk) begin
68     if (reset) begin
69         PC_ex <= 32'b0; // 复位
70         rs1Data_ex <= 32'b0;
71         rs2Data_ex <= 32'b0;
72         Imm_ex <= 32'b0;
73         rdAddr_ex <= 5'b0;
74         rs1Addr_ex <= 5'b0;
75         rs2Addr_ex <= 5'b0;
76         ALUCode_ex <= 4'b0;
77         ALUSrcA_ex <= 1'b0;
78         ALUSrcB_ex <= 2'b0;
79         MemRead_ex <= 1'b0;
80         MemWrite_ex <= 1'b0;
81         RegWrite_ex <= 1'b0;
82         MemtoReg_ex <= 1'b0;
83     end else begin
84         PC_ex <= PC_id; // 更新
85         rs1Data_ex <= rs1Data_id;
86         rs2Data_ex <= rs2Data_id;
87         Imm_ex <= Imm_id;
88         rdAddr_ex <= rdAddr_id;
89         rs1Addr_ex <= rs1Addr_id;
90         rs2Addr_ex <= rs2Addr_id;
91         ALUCode_ex <= ALUCode_id;
92         ALUSrcA_ex <= ALUSrcA_id;
93         ALUSrcB_ex <= ALUSrcB_id;
94         MemRead_ex <= MemRead_id;
95         MemWrite_ex <= MemWrite_id;
96         RegWrite_ex <= RegWrite_id;
97         MemtoReg_ex <= MemtoReg_id;
98     end
99 end
100 endmodule
101
102 // EXMEM寄存器模块
103 module EX_MEM (
104     input clk, // 时钟信号
105     input MemWrite_ex, // 输入内存写信号
106     input RegWrite_ex, // 输入寄存器写信号
107     input MemtoReg_ex, // 输入内存到寄存器信号
108     input [31:0] ALUResult_ex, // 输入ALU计算结果
109     input [31:0] MemWriteData_ex, // 输入内存数据
110     input [4:0] rdAddr_ex, // 输入目的寄存器地址
111     input reset,
112     output reg MemWrite_mem, // 输出内存写信号
113     output reg RegWrite_mem, // 输出寄存器写信号
114     output reg MemtoReg_mem, // 输出内存到寄存器信号
115     output reg [31:0] ALUResult_mem, // 输出ALU计算结果
116     output reg [31:0] MemWriteData_mem, // 输出内存数据
117     output reg [4:0] rdAddr_mem // 输出目的寄存器地址
118 );
119
120
121
122
123

```

```

124
125     always @(posedge clk) begin
126         if (reset) begin
127             MemWrite_mem <= 0;           // 更新
128             RegWrite_mem <= 0;
129             MemtoReg_mem <= 0;
130             ALUResult_mem <= 0;
131             MemWriteData_mem <= 0;
132             rdAddr_mem <= 0;
133         end else begin
134             MemWrite_mem <= MemWrite_ex;   // 更新
135             RegWrite_mem <= RegWrite_ex;
136             MemtoReg_mem <= MemtoReg_ex;
137             ALUResult_mem <= ALUResult_ex;
138             MemWriteData_mem <= MemWriteData_ex;
139             rdAddr_mem <= rdAddr_ex;
140         end
141     end
142
143 endmodule
144
145
146 // MEMWB寄存器模块
147 module MEM_WB (
148     input clk,                // 时钟信号
149     input RegWrite_mem,       // 输入寄存器写信号
150     input MemtoReg_mem,       // 输入内存到寄存器信号
151     input [31:0] ALUResult_mem, // 输入ALU计算结果
152     input [31:0] MemDout_mem, // 输入内存数据
153     input [4:0] rdAddr_mem,   // 输入目的寄存器地址
154     input reset,
155
156     output reg RegWrite_wb,    // 输出寄存器写信号
157     output reg MemtoReg_wb,    // 输出内存到寄存器信号
158     output reg [31:0] ALUResult_wb, // 输出ALU计算结果
159     output reg [31:0] MemDout_wb, // 输出内存数据
160     output reg [4:0] rdAddr_wb // 输出目的寄存器地址
161 );
162
163 // 在时钟上升沿时触发
164 always @(posedge clk) begin
165     if (reset) begin
166         RegWrite_wb <= 0;           // 更新
167         MemtoReg_wb <= 0;
168         ALUResult_wb <= 0;
169         MemDout_wb <= 0;
170         rdAddr_wb <= 0;
171     end else begin
172         RegWrite_wb <= RegWrite_mem;   // 更新
173         MemtoReg_wb <= MemtoReg_mem;
174         ALUResult_wb <= ALUResult_mem;
175         MemDout_wb <= MemDout_mem;
176         rdAddr_wb <= rdAddr_mem;
177     end
178 end
179
180 endmodule

```

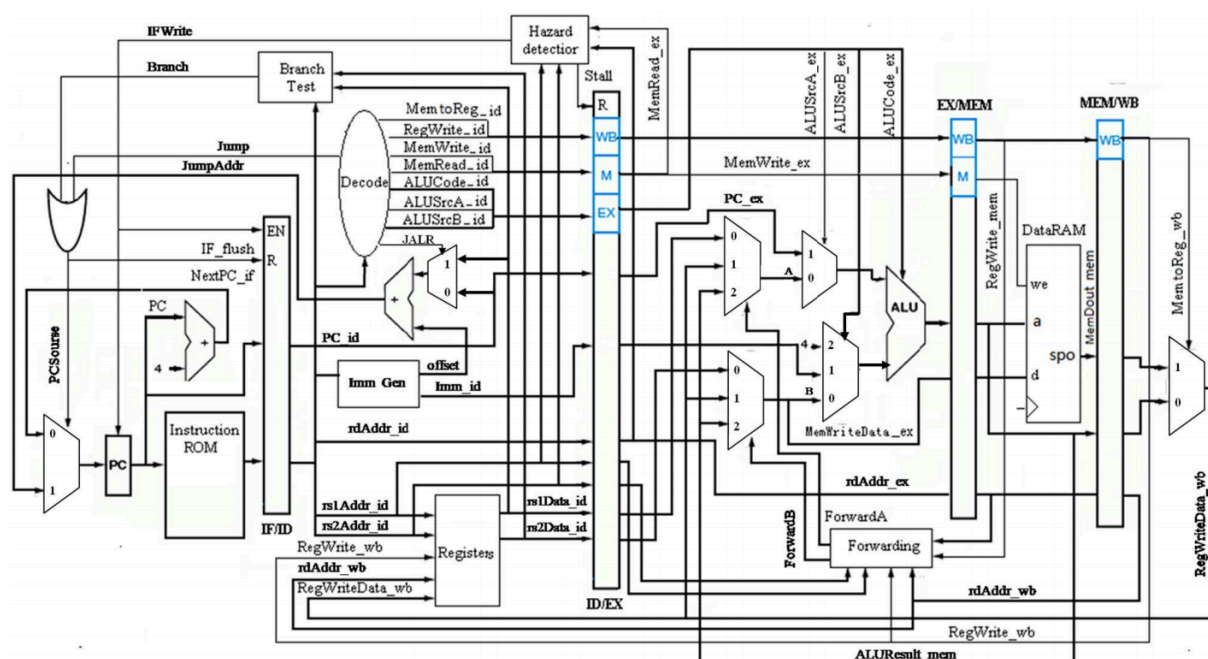
代码设计解析

四组流水线寄存器分别承担指令和控制信号在各级间的可靠传递。IF_ID 在支持 reset 清零的同时增加 EN 使能信号，用于在数据冒险时冻结 IF/ID 级；ID_EX 在 reset 时将所有控制和数据信号清零，相当于插入气泡，保证异常状态不会传播到后级；EX_MEM 和 MEM_WB 则作为普通寄存器，将 ALU 结果、访存数据及写回控制信号顺序传送到 MEM、WB 级，为顶层 CPU 模块按图连接各级并观察关键内部信号提供了结构化的时序边界。

通过在顶层将 IF_flush 与 Stall 分别接入 IF_ID 和 ID_EX 的复位与使能端，整个流水线在面对控制冒险时以清空前两级、数据冒险时以冻结前两级的方式插入气泡，使复杂的时序与数据相关在结构上都被约束到这几级寄存器上，便于分析和调试。

5.6 CPU 顶层模块

CPU 顶层结构如下所示：



按照原理框图连接各模块即可。为了测试方便，可将关键变量输出，关键变量有：指令指针 PC、指令码 Instruction_id、流水线插入气泡标志 Stall、分支标志 JumpFlag 即 Jump, Branch、ALU 输入输出（ALU_A、ALU_B、ALUResult_ex）和数据存储器的输出 MemDout_mem。

按照顺序连接各个接口和模块，最终实现整个 CPU 的顶层模块：

```
1 `timescale 1ns / 1ps
2 module Risc5CPU(clk, reset, JumpFlag, Instruction_id, ALU_A,
```

```

3          ALU_B, ALUResult_ex, PC, MemDout_mem, Stall
4          //,
5          //Imm_id_test, Imm_ex_test, ALUSrcB_ex_test
6          );
7      input clk;
8      input reset;
9      output[1:0] JumpFlag;
10     output [31:0] Instruction_id;
11     output [31:0] ALU_A;
12     output [31:0] ALU_B;
13     output [31:0] ALUResult_ex;
14     output [31:0] PC;
15     output [31:0] MemDout_mem;
16     output Stall;
17
18     //测试output
19     // output [31:0] Imm_id_test,Imm_ex_test;
20     //output [1:0] ALUSrcB_ex_test;
21
22     //IF模块
23     wire Branch, Jump, IFWrite, IF_flush;
24     wire [31:0] JumpAddr, Instruction_if;
25     assign JumpFlag[1] = Jump;
26     assign JumpFlag[0] = Branch;
27     // wire [31:0] PC;//已经在顶层定义
28
29     IF IF_inst(
30         .clk(clk),
31         .reset(reset),
32         .Branch(Branch),
33         .Jump(Jump),
34         .IFWrite(IFWrite),
35         .JumpAddr(JumpAddr),
36         .Instruction_if(Instruction_if),
37         .PC(PC),
38         .IF_flush(IF_flush)
39     );
40
41     //IFID寄存器
42     wire [31:0] PC_id;
43     // wire [31:0] Instruction_id;//已经在顶层定义
44     IF_ID IF_ID_inst(
45         .clk(clk),
46         .reset(IF_flush || reset),
47         .EN(IFWrite),
48         .Instruction_if(Instruction_if),
49         .PC_if(PC),
50         .Instruction_id(Instruction_id),
51         .PC_id(PC_id)
52     );
53
54     //ID模块
55     //输入信号
56     wire RegWrite_wb;
57     wire [4:0] rdAddr_wb;
58     wire [31:0] RegWriteData_wb;
59     wire MemRead_ex;
60     wire [4:0] rdAddr_ex;
61     //输出信号

```

```

62     wire MemtoReg_id;
63     wire RegWrite_id;
64     wire MemWrite_id;
65     wire MemRead_id;
66     wire [3:0] ALUCode_id;
67     wire ALUSrcA_id;
68     wire [1:0]ALUSrcB_id;
69     // wire Stall; //已经在顶层定义
70     wire [31:0] Imm_id;
71     wire [31:0] rs1Data_id;
72     wire [31:0] rs2Data_id;
73     wire[4:0] rs1Addr_id,rs2Addr_id,rdAddr_id;
74
75     ID ID_inst(
76         .clk(clk),
77         .Instruction_id(Instruction_id),
78         .PC_id(PC_id),
79         .RegWrite_wb(RegWrite_wb),
80         .rdAddr_wb(rdAddr_wb),
81         .RegWriteData_wb(RegWriteData_wb),
82         .MemRead_ex(MemRead_ex),
83         .rdAddr_ex(rdAddr_ex),
84         .MemtoReg_id(MemtoReg_id),
85         .RegWrite_id(RegWrite_id),
86         .MemWrite_id(MemWrite_id),
87         .MemRead_id(MemRead_id),
88         .ALUCode_id(ALUCode_id),
89         .ALUSrcA_id(ALUSrcA_id),
90         .ALUSrcB_id(ALUSrcB_id),
91         .Stall(Stall),
92         .Branch(Branch),
93         .Jump(Jump),
94         .IFWrite(IFWrite),
95         .JumpAddr(JumpAddr),
96         .Imm_id(Imm_id),
97         .rs1Data_id(rs1Data_id),
98         .rs2Data_id(rs2Data_id),
99         .rs1Addr_id(rs1Addr_id),
100        .rs2Addr_id(rs2Addr_id),
101        .rdAddr_id(rdAddr_id)
102    );
103
104    // IDEX寄存器
105    wire [31:0] PC_ex;
106    wire [31:0] rs1Data_ex;
107    wire [31:0] rs2Data_ex;
108    wire [31:0] Imm_ex;
109    // wire [4:0] rdAddr_ex; //已经在ID模块定义
110    wire [4:0] rs1Addr_ex;
111    wire [4:0] rs2Addr_ex;
112    wire [3:0] ALUCode_ex;
113    wire ALUSrcA_ex;
114    wire [1:0] ALUSrcB_ex;
115    // wire MemRead_ex; //已经在ID模块定义
116    wire MemWrite_ex;
117    wire RegWrite_ex;
118    wire MemtoReg_ex;
119
120    ID_EX ID_EX_inst(

```

```

121         .clk(clk),
122         .reset(reset || Stall),
123         .PC_id(PC_id),
124         .rs1Data_id(rs1Data_id),
125         .rs2Data_id(rs2Data_id),
126         .Imm_id(Imm_id),
127         .rdAddr_id(rdAddr_id),
128         .rs1Addr_id(rs1Addr_id),
129         .rs2Addr_id(rs2Addr_id),
130         .ALUCode_id(ALUCode_id),
131         .ALUSrcA_id(ALUSrcA_id),
132         .ALUSrcB_id(ALUSrcB_id),
133         .MemRead_id(MemRead_id),
134         .MemWrite_id(MemWrite_id),
135         .RegWrite_id(RegWrite_id),
136         .MemtoReg_id(MemtoReg_id),
137         .PC_ex(PC_ex),
138         .rs1Data_ex(rs1Data_ex),
139         .rs2Data_ex(rs2Data_ex),
140         .Imm_ex(Imm_ex),
141         .rdAddr_ex(rdAddr_ex),
142         .rs1Addr_ex(rs1Addr_ex),
143         .rs2Addr_ex(rs2Addr_ex),
144         .ALUCode_ex(ALUCode_ex),
145         .ALUSrcA_ex(ALUSrcA_ex),
146         .ALUSrcB_ex(ALUSrcB_ex),
147         .MemRead_ex(MemRead_ex),
148         .MemWrite_ex(MemWrite_ex),
149         .RegWrite_ex(RegWrite_ex),
150         .MemtoReg_ex(MemtoReg_ex)
151     );
152
153     //EX模块
154     // wire [31:0] ALUResult_ex; //已经在顶层定义
155     wire [31:0] MemWriteData_ex;
156     // wire [31:0] ALU_A; //已经在顶层定义
157     // wire [31:0] ALU_B; //已经在顶层定义
158     // wire [31:0] RegWriteData_wb; //已经在ID模块定义
159     wire [31:0] ALUResult_mem;
160     wire [4:0] rdAddr_mem;
161     // wire [4:0] rdAddr_wb; //已经在ID模块定义
162     wire RegWrite_mem;
163     // wire RegWrite_wb; //已经在ID模块定义
164
165     EX EX_inst(
166         .ALUCode_ex(ALUCode_ex),
167         .ALUSrcA_ex(ALUSrcA_ex),
168         .ALUSrcB_ex(ALUSrcB_ex),
169         .Imm_ex(Imm_ex),
170         .rs1Addr_ex(rs1Addr_ex),
171         .rs2Addr_ex(rs2Addr_ex),
172         .rs1Data_ex(rs1Data_ex),
173         .rs2Data_ex(rs2Data_ex),
174         .PC_ex(PC_ex),
175         .RegWriteData_wb(RegWriteData_wb),
176         .ALUResult_mem(ALUResult_mem),
177         .rdAddr_mem(rdAddr_mem),
178         .rdAddr_wb(rdAddr_wb),

```

```

179         .RegWrite_mem(RegWrite_mem),
180         .RegWrite_wb(RegWrite_wb),
181         .ALUResult_ex(ALUResult_ex),
182         .MemWriteData_ex(MemWriteData_ex),
183         .ALU_A(ALU_A),
184         .ALU_B(ALU_B)
185     );
186
187     //EXMEM寄存器
188     wire MemWrite_mem;
189     // wire RegWrite_mem;//已经在EX模块定义
190     wire MemtoReg_mem;
191     // wire [31:0] ALUResult_mem;//已经在EX模块定义
192     wire [31:0] MemWriteData_mem;
193     // wire [4:0] rdAddr_mem;//已经在EX模块定义
194
195     EX_MEM EX_MEM_inst(
196         .clk(clk),
197         .MemWrite_ex(MemWrite_ex),
198         .RegWrite_ex(RegWrite_ex),
199         .MemtoReg_ex(MemtoReg_ex),
200         .ALUResult_ex(ALUResult_ex),
201         .MemWriteData_ex(MemWriteData_ex),
202         .rdAddr_ex(rdAddr_ex),
203         .reset(reset),
204         .MemWrite_mem(MemWrite_mem),
205         .RegWrite_mem(RegWrite_mem),
206         .MemtoReg_mem(MemtoReg_mem),
207         .ALUResult_mem(ALUResult_mem),
208         .MemWriteData_mem(MemWriteData_mem),
209         .rdAddr_mem(rdAddr_mem)
210     );
211
212     //MEM模块
213     wire [31:0] MemDout_mem;
214     DataRAM DataRAM_inst(
215         .a(ALUResult_mem[7:2]),
216         .d(MemWriteData_mem),
217         .clk(clk),
218         .we(MemWrite_mem),
219         .spo(MemDout_mem)
220     );
221
222     //MEMWB寄存器
223     // wire RegWrite_wb;//已经在ID模块定义
224     wire MemtoReg_wb;
225     wire [31:0] ALUResult_wb;
226     wire [31:0] MemDout_wb;
227     // wire [4:0] rdAddr_wb;//已经在ID模块定义
228
229     MEM_WB MEM_WB_inst(
230         .clk(clk),
231         .RegWrite_mem(RegWrite_mem),
232         .MemtoReg_mem(MemtoReg_mem),
233         .ALUResult_mem(ALUResult_mem),
234         .MemDout_mem(MemDout_mem),
235         .rdAddr_mem(rdAddr_mem),
236         .reset(reset),
237         .RegWrite_wb(RegWrite_wb),

```

```

238         .MemtoReg_wb(MemtoReg_wb),
239         .ALUResult_wb(ALUResult_wb),
240         .MemDout_wb(MemDout_wb),
241         .rdAddr_wb(rdAddr_wb)
242     );
243
244     //WB模块
245     assign RegWriteData_wb=MemtoReg_wb?MemDout_wb:ALUResult_wb;
246
247     //测试信号赋值
248     assign Imm_id_test=Imm_id;
249     assign ALUSrcB_ex_test=ALUSrcB_ex;
250     assign Imm_ex_test=Imm_ex;
251
252 endmodule

```

六 实验结果

6.1 仿真结果

使用如下的代码进行仿真测试，CPU 将会运行 instructionsROM 中的指令，将仿真得到的信号值与预期值进行比较，以验证 CPU 的正确性。

```

1  initial
2  begin
3      // Initialize Inputs
4      clk = 0;
5      reset = 1;
6      // Wait 100 ns for global reset to finish
7      #100
8      #51 reset=0;
9      #2200 $finish;
10 end
11 always #20 clk=~clk;

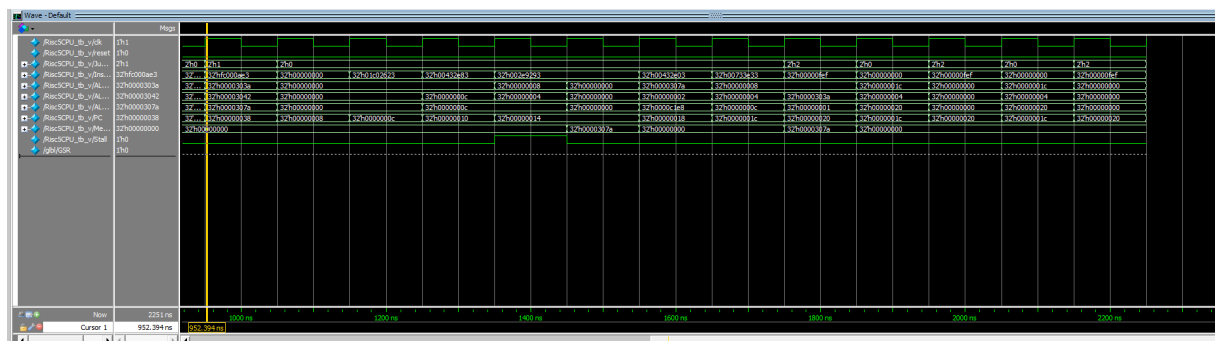
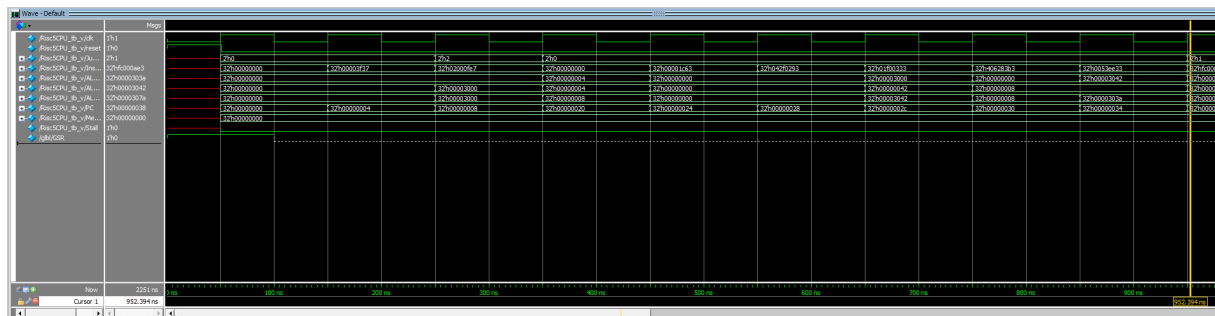
```

预期结果在下表 30.11 中已经给出。

表 30.11 测试程序的运行结果

reset	clk	PC	Instruction (ID)	JumpFlag	Stall	ALU_A	ALU_B	ALUResult	MemDout (MEM)
1	1	0	0	0	0	0	0	0	-
0	2	4	00003f37 (lui)	0	0	-	-	-	-
	3	8	02000fe7 (jalr)	2	0	-	3000	3000	-
	4	20	0	0	0	4	4	8	-
	5	24	00001c63 (bne)	0	0	--	-	-	-
	6	28	042f0293 (addi)	0	0	-	-	-	-
	7	2c	01f00333 (add)	0	0	3000	42	3042	-
	8	30	406283b3 (sub)	0	0	0	8	8	-
	9	34	0053ee33 (or)	0	0	3042	8	303a	-
	10	38	fc000ae3 (beq)	1	0	303a	3042	307a	-
	11	8	0	0	0	-	-	-	-
	12	c	01c02623 (sw)	0	0	-	-	-	-
	13	10	00432e83 (lw)	0	0	0	c	c	-
	14	14	002e9293 (sll)	0	1	8	4	c	-
	15			0	0	-	-	-	307a
	16	18	00432e03 (lw)	0	0	307a	2	c1e8	-
	17	1c	00733e33 (sltu)	0	0	8	4	c	-
	18	20	0000fef (jal)	2	0	8	303a	1	307a
	19	1c	0	0	0	1c	4	20	-
	20	20	0000fef (jal)	2	0	-	-	-	-
	21	1c	0	0	0	1c	4	20	-

得到的仿真结果如下：

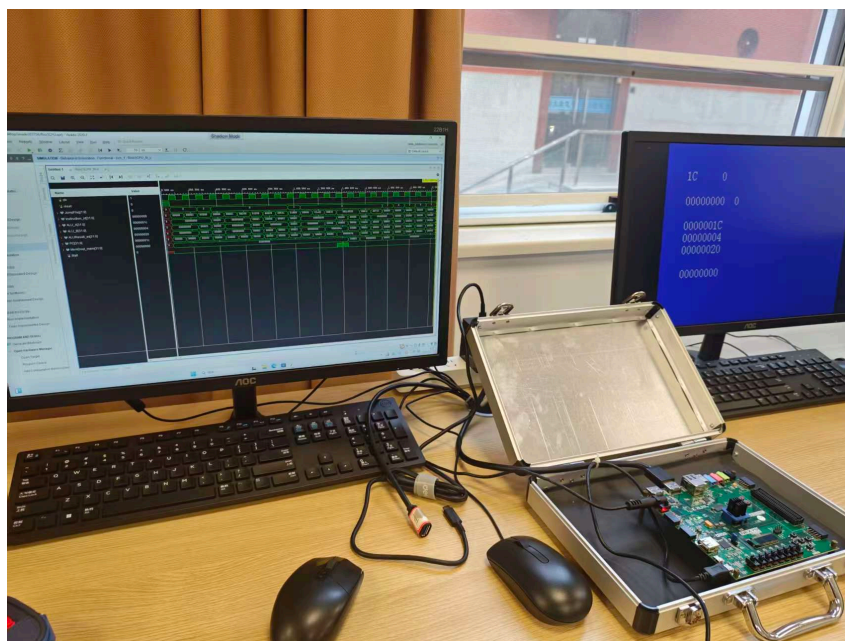


对比各个信号的电平变化，发现结果一致，说明我们的设计无误。

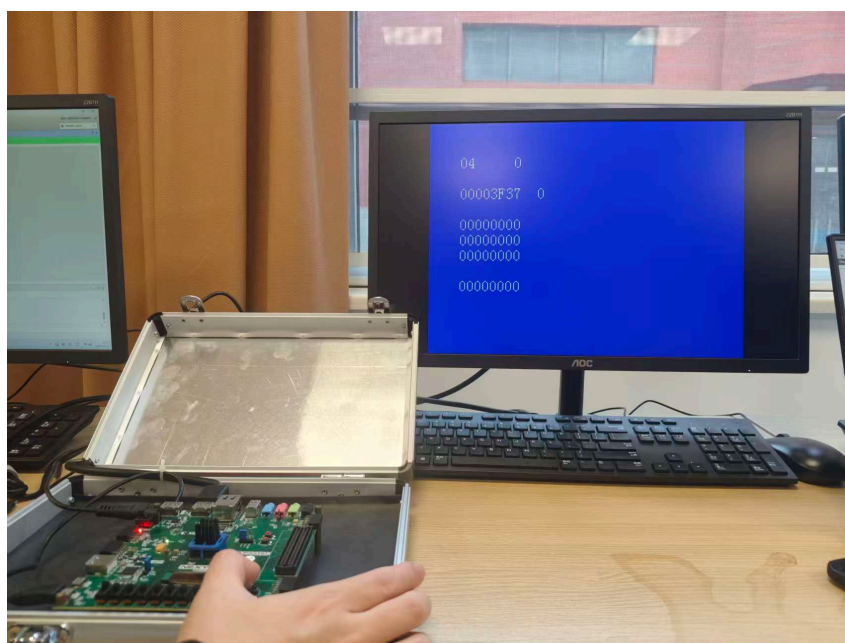
6.2 硬件测试结果

再次打开 Vivado 文件夹下的 Risc5CPU.xpr 工程，添加流水线 CPU 设计的全部代码，然后综合、实现和下载至 Nexys Video 开发板。连接带有 HDMI 接口的显示器，进行测试。首先将 SW0 置于低电平，使 RISC-V CPU 工作在“单步”运行模式。复位后，每按一下上边按键，RISC-V CPU 运行一步，记录下显示器上的结果，对照 CPU 行为级仿真预期结果验证设计是否正确。

给开发板上电、连数据线、HDMI 线，最后的硬件连接如下：



将程序烧到开发板上按下按钮，显示器画面如下：



由于篇幅原因，这里只展示其中一步。

按下上边按键，RISC-V CPU 运行一步，显示器画面发生一次变化。对照表 30.11 的结果和旁边显示器的仿真结果可以对照仿真结果与实际结果，发现程序可以正常运行。反复按按钮直到程序进入循环，如果结果均一致则说明设计大功告成。

由于开发板按钮没有做消抖功能，可能会出现显示器页面连续变化的情况。这时只能按下中间+上边按钮使得程序归零，从头重新运行观察。

七 思考题

如下两条指令，条件分支指令试图读取上一条指令的目标寄存器，插入气泡或数据转发都无法解决流水线冲突问题。为什么在大多 CPU 架构中，都不去解决这一问题？这一问题应在什么层面中解决？

```
1 lw x28, 04(x6)
2 beq x28, x29, Loop
```

lw 指令从内存中加载数据到寄存器 x28，而下一条 beq 指令需要使用 x28 的值进行条件判断。由于 lw 指令的结果还未写回寄存器，beq 指令无法立即获取到正确的 x28 值，导致数据冒险。通常不直接在硬件层面解决这种问题的原因如下：

- (1) **原因：**在硬件层面解决这种问题需要增加额外的逻辑电路来处理数据冒险。在此案例中，需要单独设计一个数据前推模块，对分支指令相关问题进行处理和连线。这需从各个寄存器连接到分支检测模块，会引入复杂的转发逻辑和流水线暂停机制，可能导致数据通路过长，引入更多的干扰，带来更高的成本。即使在硬件层面解决了这种问题，也会引入流水线暂停，并没有显著提高运行速度。
- (2) **解决方法：**对于分支问题，现代编译器可以通过指令调度和优化技术来避免这种数据冒险问题，使得解决问题的成本大大降低。例如，编译器可以在 lw 和 beq 指令之间插入其他无关指令，或者重新安排这些指令的顺序以避免数据冒险。此外，程序员也可以通过手动插入 NOP 指令或重新安排指令顺序来避免数据冒险。

综上所述，大多数 CPU 架构选择在软件层面解决这种数据冒险问题，而不是在硬件层面直接处理，因为修改硬件的收益并不显著，而且从软件层面解决成本更低。